Agente IA 4.0.2

Documentación técnica

Arquitectura, plataformas, administración y API

Documentación técnica

Arquitectura
- Estructura
- Flujo de un mensaje
- Perfiles de hardware
  - Autoajuste en cada arranque

Plataformas, instaladores y empaquetado
- Matriz de compatibilidad
- Qué hace cada instalador
  - Opciones de los scripts de instalación
- Instalar desde el código fuente (sin compilar)
  - Aceleración incluida en los instaladores
  - Limpieza al final de la instalación
  - AppImage y Flatpak
- Servicio y descargas para la oficina
- Empaquetado
- Apps Android (versión 2.2.0)
  - Compilar la app Nexus-Office VPN
  - Compilar la app Nexus-Office autónomo
- Portabilidad
- Acceso desde otros dispositivos (móvil, tablet, consola, TV)

Motor de IA y acelerador (CPU, GPU o NPU)
- Dónde se elige
- Perfiles de hardware (2.2)
- Equipos con dos o más GPU y Linux (2.2)
- Motores
- Aceleradores
- Cómo se comprueba que acelera de verdad
- Runtimes incluidos por instalador
- Configuración

Motores de inferencia por hardware
- Motores
- Android (APK autónoma), desde la 4.0.2
- Qué motor va con cada chip
- El banco manda sobre la tabla
- Aceleradores y caídas
- Modelos para cada motor
- Pruebas

Nexus-Office Autónomo para Android
- CPU, GPU y NPU
- Se optimiza sola
- Cómo funciona
- Conectar a mi ordenador (desde 2.2)
- Compilar
- Notas

Guía de administrador / desarrollador
- Qué desbloquea
- Configuración destacada (data/config.yaml)
- Acceso remoto por internet
- Autoajuste
- Runtimes y aceleración
- Línea de comandos

- Modelo de IA elegido al instalar
- Servicio (arranque automático)
- Descargas para la oficina
- Construir los instaladores
- Datos

Nexus-Office como servicio: arranque automático
- Cómo se activa
- Comandos
- Qué hace en cada sistema
- App Nexus-Office autónomo (Android)
- Quitar el servicio
- Registro (logs)
- Problemas frecuentes

API
- WebSocket /ws
- REST principal
- Servicio y descargas
- Gestor de conocimiento
- Acceso remoto

Herramientas: sistema modular y supervisado
- Anatomía de una herramienta
- Supervisión
- Herramientas incluidas
- Propuestas de los agentes

Memoria y gestor de conocimiento
- Cómo se organiza
  - El organizador
- Gestor de conocimiento (Ajustes, modo administrador)
  - Formatos de grafo que se pueden importar
  - Leer un ZIP (también RAR, 7z y TAR)
- Modelo de embeddings: Qwen3-Embedding 0.6B
  - Configuración (data/config.yaml)
- API

Importar grafos como conocimiento
- Cómo se importa
  - Desde la app
  - Desde la terminal
  - Desde el chat
  - Dentro de un ZIP
- Formatos admitidos
- Qué se guarda de cada grafo
- Para desarrolladores

Biblioteca RAG y conversión URL → PDF
- RAG (generación aumentada por recuperación)
- Leer páginas como un navegador (2.2)
- Grafo general o grafo de un agente
- Archivos ZIP, RAR, 7z y TAR con proyectos o documentación
- Rendimiento: procesador al máximo
- Rastreo (crawling) de un sitio de documentación
  - Todo el dominio, sin excepciones (2.0)
  - Dominios enteros: rastreo por lotes
- URL → PDF estructurado → conocimiento

Red cooperativa (LAN virtual por internet)
- Cómo funciona

Servicios
Puesta en marcha
Hacer accesible un nodo
VPN cifrada para la app Nexus-Office VPN (WireGuard)
Puesta en marcha
API (solo administrador)
Comprobado
Problemas frecuentes
Acceso remoto por internet
Cómo funciona
El cifrado elegido y por qué
Asistente del router (lo más fácil)
Puesta en marcha (modo administrador)
Emparejar un dispositivo
Avanzado
Dispositivos: luces, reloj y avatar de bolsillo
Grabar el firmware
Luces NexusLED
Reloj NexusWatch
Avatar NexusPico
Modos
API
Qué no se ha podido probar sin el hardware

Arquitectura

```
flowchart LR
  UI[webui · PWA] -- REST/WebSocket --> API[server/app.py]
  API --> AG[agents · AgentManager]
  AG --> LLM[llm · LLMManager]
  AG --> SUP[tools · Supervisor]
  AG --> LRN[memory · Learner]
  AG --> LNG[agents · Lounge]
  LLM --> LS[llama-server · runtime nativo]
  LLM --> OL[Ollama / OpenAI-compatible / Transformers]
  LLM --> BI[núcleo integrado sin modelo]
  LLM -. peer:nodo .-> COOP[coop · CoopNode]
  SUP --> TB[herramientas integradas]
  SUP --> TC[herramientas de la comunidad · sandbox]
  LRN --> MEM[(SQLite: memoria, grafo, auditoría, salón)]
  HW[hardware · detección/perfil/monitor/tuner] --> LLM
  HW --> MED[media · imagen/música/voz]
  MM[models · gestor 5 repositorios] --> LLM
  MM --> MED
```

Estructura

Ruta	Responsabilidad
`nexus/paths.py`	Rutas portables relativas al proyecto (`NEXUS_DATA_DIR` para redirigir los datos).
`nexus/config.py`	Configuración en dos niveles (`user.*` editable en modo Usuario; el resto solo Admin) y PIN PBKDF2.
`nexus/hardware/`	`detect.py` (CPU/RAM/GPU/NPU/runtimes), `android_accel.py` (CPU/GPU/NPU del móvil), `profiles.py` (perfil óptimo), `monitor.py` (muestras, latido, resumen de sesión), `tuner.py` (autoajuste).
`nexus/runtimes.py`	Descarga y localiza binarios oficiales de llama.cpp y stable-diffusion.cpp por variante (cpu, cuda, rocm, vulkan, sycl, openvino, metal).
`nexus/service.py`	Arranque automático como servicio: systemd, cron `@reboot`, launchd, Programador de tareas de Windows y termux-services (`python -m nexus servicio`, ver SERVICIO.md).
`nexus/downloads.py`	Instaladores de `<datos>/descargas/` clasificados por plataforma para *Sistema → Descargas*.
`nexus/proc.py`	Subprocesos ligados a la vida de Nexus-Office (Job Object en Windows, PDEATHSIG en Linux).
`nexus/llm/`	Backends (`llamaserver`, `llamacpp`, `ollama`, `openai`, `transformers`, `builtin`), protocolo de herramientas, recorte de contexto.
`nexus/memory/`	Embeddings (sentence-transformers › Ollama › hashing), `MemoryStore`, `KnowledgeGraph`, `Learner`.
`nexus/agents/`	`AgentManager` (enrutado, prompt, bucle de herramientas, delegación) y `Lounge` (salón virtual).
`nexus/tools/`	Registro modular, supervisor, ejecutor aislado y propuestas con análisis AST.
`nexus/models/`	Catálogo, lector GGUF, 5 fuentes, recomendaciones por hardware.
`nexus/media/`	Imagen, música y voz con degradación elegante.
`nexus/coop/`	Cifrado, invitaciones y nodo de la LAN virtual.
`nexus/server/`	API REST + WebSocket.
`nexus/remote/`	Acceso remoto: certificado propio y TLS 1.3 (`tlscert`), dispositivos y códigos de emparejamiento (`devices`), UPnP (`upnp`) y `RemoteGate`, la puerta ASGI del puerto HTTPS que exige la llave del dispositivo en HTTP y WebSocket.
`nexus/web/`	Interfaz (HTML/CSS/JS sin dependencias de compilación), avatar 3D y salón en canvas, PWA. `js/device.js` detecta el cliente (app Android, PWA o navegador) y el dispositivo antes del primer pintado y marca `<html>` con `data-layout` (phone/tablet/desktop), `data-orient`, `data-input` y `data-lowpower`; el CSS adapta el diseño con esos atributos.

Flujo de un mensaje

1. La UI envía `{"type":"chat"}` por WebSocket.
2. `AgentManager.run` elige agente (palabras clave + habilidades aprendidas en el salón), el `Learner` extrae hechos del mensaje y construye el contexto (recuerdos, hechos del grafo, ejemplos valorados, lecciones).
3. **Vía rápida**: intenciones inequívocas (crear imagen/música, calcular, hora) se ejecutan directamente con su herramienta.

4. Si no, el LLM responde en streaming. Con `llama-server`/OpenAI se usan **llamadas nativas a herramientas**; con otros backends, un protocolo de texto `<tool>{…}</tool>` compatible con cualquier modelo.
5. Cada herramienta pasa por el **Supervisor** (validación, política por riesgo, aprobación, límite de tiempo, sandbox, auditoría).
6. La respuesta se guarda, el grafo se actualiza (el avatar lo refleja) y el salón registra la tarea.

Perfiles de hardware

Hardware	Acelerador	Ejemplo de decisión
NVIDIA	`cuda`	Todas las capas en GPU, modelo según VRAM (≥20 GB → 32B), flash attention, Whisper en GPU, MusicGen.
AMD dedicada	`vulkan` (Windows) / `rocm` (Linux)	Capas en GPU, PyTorch DirectML para imagen/música en Windows.
Intel Arc	`vulkan` / `xpu` / `sycl`	Capas en GPU.
Intel/AMD integrada	`cpu` (con `openvino`/`vulkan` como alternativas medibles)	La CPU suele ganar en modelos pequeños.
Apple Silicon	`metal`	Memoria unificada, ANE como NPU.
Solo CPU	`cpu`	Modelo según RAM y núcleos (2 núcleos → ≤3B), hilos = núcleos físicos.
NPU	`openvino` / `onnxruntime-qnn` / Ryzen AI / CoreML	Descarga de embeddings y voz cuando el runtime está disponible.
Android (app autónoma)	`hexagon` (NPU de Snapdragon 8 Gen 2+) / `opencl` (Adreno) / `vulkan` (Mali, Immortalis, Xclipse, PowerVR) / `cpu`	`hardware/android_accel.py` detecta SoC, GPU y NPU (con `libllama-server-gpu.so --list-devices`: `HTP0`, `GPUOpenCL`, `Vulkan0`); orden NPU → GPU → CPU, con vuelta automática si una falla al arrancar; el autoajuste compara el acelerador con la CPU con respuestas reales. Las NPU de MediaTek, Tensor y Exynos se informan pero llama.cpp no las usa.

Autoajuste en cada arranque

El monitor guarda el resumen de cada sesión (tokens/s, latencia, RAM/VRAM pico, errores) y un latido periódico. Al arrancar, el *tuner*:

- reduce contexto/lote/capas GPU si hubo **falta de memoria** (no si simplemente se cerró la aplicación);
- recupera progresivamente el perfil base tras sesiones estables;
- amplía el contexto si sobra RAM;
- **explora** distintos números de hilos y se queda con el más rápido medido;
- sugiere un modelo más pequeño si la velocidad es baja.

Mínimos de seguridad: contexto ≥ 2048 y lote ≥ 128.

Plataformas, instaladores y empaquetado

Matriz de compatibilidad

Todos los instaladores son **100 % portables**: llevan Python, las dependencias (wheels) y llama-server/sd-cli, y se instalan sin internet. Solo el modelo de IA se descarga, y solo si hay conexión. Cómo instalar cada uno, paso a paso: Guía de usuario → Instalación.

Plataforma	Instalador	Ejecución local de IA	Notas
Windows 10/11 x64	`Nexus-Office-Setup-2.2.0-windows-x64.exe`	✅ CPU, CUDA (NVIDIA), Vulkan (NVIDIA, AMD, Intel), OpenVINO	Instalador gráfico NSIS, accesos directos, desinstalador en *Aplicaciones*, servicio opcional. **Probado en el equipo de desarrollo.**
Windows 11 ARM64 (Snapdragon X)	`Nexus-Office-Setup-2.2.0-windows-arm64.exe`	✅ CPU ARM64 y Vulkan (Adreno); NPU Hexagon vía `onnxruntime-qnn`	llama-server y sd-cli nativos ARM64; Python x64 emulado por Windows 11. El instalador se niega a seguir en un Windows que no sea ARM64.
Linux x86_64	`Nexus-Office-2.2.0-linux-x86_64.run`, `nexus-office_2.2.0_amd64.deb`, Docker	✅ CPU, CUDA, Vulkan, OpenVINO	Ubuntu, Debian, Fedora, Arch... (glibc 2.28 o posterior).
Steam Deck (SteamOS)	`Nexus-Office-2.2.0-linux-x86_64.run`	✅ CPU / Vulkan (APU AMD)	Instala en modo escritorio en tu carpeta de usuario (SteamOS es de solo lectura) y añade `start.sh` como *juego no-Steam*.
Linux ARM64 (Raspberry Pi 4/5, Jetson, Orange Pi)	`Nexus-Office-2.2.0-linux-aarch64.run`, `nexus-office_2.2.0_arm64.deb`	✅ CPU y Vulkan (Pi 5: Qwen3-VL 2B Thinking); CUDA en Jetson Orin	Hace falta un sistema de **64 bits** (Raspberry Pi OS 64-bit, Ubuntu). Los sistemas de 32 bits (`armv7l`) no están soportados. Usa disco SSD/USB3 para los modelos.
macOS Apple Silicon / Intel	`Nexus-Office-2.2.0-macos-arm64.tar.gz`, `Nexus-Office-2.2.0-macos-x86_64.tar.gz`	✅ Metal (Apple Silicon), CPU	Incluye `Nexus-Office.command` y `Nexus-Office.app`. Primera apertura: clic derecho → Abrir (no está firmado).
Android (app Nexus-Office VPN)	`Nexus-Office-2.2.0-android-vpn.apk`	⚙️ Cliente del PC	App nativa (Android 7+). Conecta con el Nexus-Office de tu PC (Windows, Linux o macOS) por **VPN WireGuard cifrada** desde cualquier sitio, por HTTPS con certificado anclado o por la Wi-Fi de la oficina. Webui completa, voz del sistema, adjuntar y descargar ficheros. Ver VPN.md.
Android (app Nexus-Office, recomendada)	`Nexus-Office-2.2.0-android-autonomo.apk`	✅ CPU (3 variantes ARM), GPU (Adreno por OpenCL; Mali, Immortalis, Xclipse y PowerVR por Vulkan) y NPU Hexagon de los Snapdragon 8 Gen 2 o posteriores (llama.cpp Hexagon, experimental). Otras NPU (APU, TPU, Exynos) detectadas pero sin uso	Nexus-Office 100 % en el móvil, sin PC (Android 9+, ARM64). La APK lleva Python 3.12, Nexus-Office, sus dependencias y llama.cpp (CPU y GPU); detecta CPU, GPU y NPU, mide si el modelo va más rápido en GPU o en CPU y solo descarga el modelo de IA adecuado a la RAM del móvil (Qwen3-VL 2B o 4B Thinking). Arranque al encender opcional. También se conecta al Nexus-Office del PC (*RED → Conectar a mi ordenador*). Ver android-autonomo/README.md.
Android ARM64 (Termux)	Desde el código fuente o desde la app Nexus-Office VPN	✅ CPU vía Termux (`pkg llama-cpp`)	Ya no se entrega un paquete para Termux, pero `install-termux.sh` sigue en el código fuente, y la app Nexus-Office VPN da el comando que descarga Nexus-Office desde tu PC (`/api/package/termux`). Instala Termux desde F-Droid. Para la mayoría de móviles es más sencilla la app Nexus-Office autónomo.
iPhone / iPad	PWA	⚠️ Cliente	iOS no permite ejecutar este tipo de servidor; úsalo como app web conectada a tu PC o a tu red cooperativa.
Xbox Series	PWA (Microsoft Edge)	⚠️ Cliente	Las consolas son plataformas cerradas: no permiten instalar Python ni runtimes. Abre la UI de tu PC desde Edge (puede instalarse como app).
PlayStation 5	Navegador	⚠️ Cliente	Sin instalación posible; acceso por navegador a un Nexus-Office de la red.

Plataforma	Instalador	Ejecución local de IA	Notas
Nintendo Switch	—	❌	No dispone de un navegador general utilizable.
Servidores / NAS / nube	`Dockerfile` (amd64/arm64)	✅	`docker run -p 8775:8775 -v nexus-data:/app/data nexus-office`

> **Por qué las consolas son "cliente"**: PlayStation, Xbox y Switch solo ejecutan software firmado por el fabricante. Nexus-Office sí funciona en ellas como interfaz (PWA) mientras la IA corre en un PC, Steam Deck, servidor o en tu red cooperativa — que además permite que varios equipos compartan el modelo más potente.

Qué hace cada instalador

Instalador	Cómo instala	Opciones
`.exe` (Windows, NSIS)	Copia la carpeta portable a `%LOCALAPPDATA%\Nexus-Office` (o la elegida) sin pedir administrador y ejecuta `install.ps1 -Offline -Clean -Modelos no` (más `-Oficina`). Después, `python -m nexus modelos --descargar` y, si se pidió, `python -m nexus servicio instalar [--red]` con UAC solo para ese paso. Accesos directos con `pythonw.exe -m nexus run` (sin consola) y *Nexus-Office (consola)* con `start.bat`.	Página de opciones: perfil de oficina, descargar el modelo, servicio al encender, acceso desde la red local, acceso directo en el escritorio. Silencioso: `/S /OFICINA=<perfil> /SERVICIO /RED /MODELOS=auto|todos|no /D=<carpeta>` (`/D` siempre al final).
`.run` (Linux)	Script `sh` con el `.tar.gz` incrustado. Lo extrae en la carpeta indicada (por defecto `~/.local/share/nexus-office`) y ejecuta `install.sh` con el resto de opciones.	`sh Nexus-Office-2.2.0-linux-<arch>.run [carpeta] [--oficina=<perfil>] [--servicio [--red]] [--modelos=auto|todos|no]`
`.deb` (Debian, Ubuntu, Raspberry Pi OS 64 bits)	Programa en `/opt/nexus-office`; `postinst` prepara `venv/` sin internet (`install.sh --offline --no-clean --modelos=no`). Lanzador `/usr/bin/nexus-office` y entrada de menú. Los datos de cada usuario van a `~/.local/share/nexus-office` (`NEXUS_DATA_DIR`). La primera ejecución de `nexus-office` evalúa el hardware y descarga el modelo. `prerm` quita el servicio.	`nexus-office [servicio|modelos|oficina|detect|pull|chat] …`; servicio con `sudo nexus-office servicio instalar [--red]`.
`.tar.gz` (macOS)	Carpeta portable. `Nexus-Office.command` ejecuta `install.sh` la primera vez (quita la cuarentena de Gatekeeper a los binarios incluidos) y después `start.sh`.	`./install.sh --oficina=<perfil> --servicio [--red] --modelos=…`

Con el **instalador gráfico de Windows** y con `install.sh`/`install.ps1`, el último paso es `python -m nexus modelos`: evalúa el hardware, elige el modelo de cada tipo y lo descarga si hay internet (ver Guía de usuario). Sin internet solo muestra la recomendación.

Opciones de los scripts de instalación

`install.sh` (Linux, macOS)	`install.ps1` / `install.bat` (Windows)	Qué hace
`--oficina=<perfil>`	`-Oficina <perfil>`	Perfil de oficina (también `NEXUS_OFICINA`). Ver OFICINA.md.
`--modelos=auto|todos|no`	`-Modelos auto|todos|no`	`auto` (por defecto): modelo de chat y visión, de documentos (embeddings) y de voz; `todos`: también el de imagen; `no`: solo muestra la recomendación. También `NEXUS_MODELOS`.
`--servicio`	`-Servicio`	Instala el arranque automático (también `NEXUS_SERVICIO=1`). Ver SERVICIO.md.
`--red`	`-Red`	Con el anterior: el servicio escucha en `0.0.0.0`.
`--offline`	`-Offline`	No usa internet: todo sale de `runtime/` y `wheelhouse/`.
`--fetch-wheels`	`-FetchWheels`	Descarga antes `wheelhouse/` y el Python a `runtime/`.
`--clean` / `--no-clean`	`-Clean` / `-NoClean`	Limpieza final sin preguntar / sin limpieza.
`--skip-extras`, `--force`	`-SkipExtras`, `-Force`	Sin extras por hardware / reinstala el runtime.

`install-termux.sh` admite `--oficina=<perfil>`, `--modelos=auto|no` y `--servicio` (en Termux el servicio escucha siempre en la red local).

Instalar desde el código fuente (sin compilar)

Desde `Nexus-Office-2.2.0-codigo-fuente.zip`, `install.sh` (Linux, WSL, macOS) e `install.bat` / `install.ps1` (Windows) descargan el Python portable y las dependencias. Si la carpeta tiene `runtime/` y `wheelhouse/` (como los instaladores), no usan internet. Wheels comprobadas para CPython 3.11 (`nexus wheels --check`, simulando la instalación sin red):

Plataforma	Base (`requirements.txt`)	Extras sin wheel (no se instalan; Nexus-Office usa la alternativa)	wheelhouse
Linux x86_64 (glibc 2.28+)	✅	—	~240 MB
Linux ARM64 (Raspberry Pi, Jetson)	✅	—	~210 MB
Windows x64 (también el instalador ARM64, con Python x64 emulado)	✅	—	~215 MB
Windows ARM64 nativo (solo instalando desde el código)	✅ con `vendor/wheels/` (PyYAML puro)	piper-tts, faster-whisper, openvino	~40 MB
macOS Apple Silicon	✅	—	~135 MB
macOS Intel	✅	piper-tts, faster-whisper, onnxruntime	~90 MB
Android (Termux)	sale de `pkg` (ya compilado) y pip solo pone Python puro	—	—

PyYAML no publica wheel para Windows ARM64. `vendor/wheels/` trae la misma versión en puro Python, y pip solo la usa donde no hay binaria. Sin voz neuronal, Nexus-Office usa la voz del sistema (pyttsx3) o la del navegador. Termux no puede usar wheels de PyPI, y si a tu repositorio le falta algún paquete, pip lo compila. Sin compilar nada en el móvil está la APK **Nexus-Office autónomo**, que ya lleva Python y sus dependencias.

`install.sh` instala siempre **wheels binarias** para el Python portable 3.11 (`--only-binary=:all:`), así que nunca compila: no necesita `gcc`, `build-essential` ni `python3-dev`. Si falta la wheel de algún paquete, se para con un mensaje claro en vez de intentar compilarlo (`NEXUS_ALLOW_BUILD=1` lo permite si de verdad hace falta). Solo admite `x86_64` y `aarch64`: en una Raspberry Pi con sistema de 32 bits se para con «Arquitectura no soportada».

La carpeta `wheelhouse/` guarda las wheels para instalar sin internet: las de `requirements.txt` y las de los extras de voz y NPU (piper-tts, faster-whisper, pyttsx3, onnxruntime y openvino). Lleva las variantes para glibc 2.39 y 2.28, así que vale desde Ubuntu 20.04 o Debian 10 en adelante. Con `wheelhouse/`, `install.sh` y `nexus setup` (también desde *Sistema*) instalan primero desde ella sin red. Si le falta algo y hay internet, lo completan desde PyPI.

```
bash install.sh --fetch-wheels      # con internet: baja wheelhouse/ (~240 MB) y el Python a runtime/, e instala
desde ellos
bash install.sh --offline           # sin internet: todo sale de runtime/ y wheelhouse/; si falta algo, lo dice
venv/bin/python3 -m nexus wheels --target linux-aarch64   # wheels para otra plataforma (linux, windows-x64, macos-
arm64, all...)
venv/bin/python3 -m nexus wheels --check                  # comprueba que wheelhouse/ instala todo sin internet
```

`wheelhouse/SHA256SUMS` permite detectar wheels dañadas al copiarlas. Los binarios de llama.cpp y stable-diffusion.cpp no son wheels: los instaladores los traen ya compilados en `bundled/runtimes/` (llama-server y sd-cli), también para GPU, y `nexus setup` no los descarga.

WSL: instala Nexus-Office en tu carpeta de Linux (`~/Nexus-Office`), no en `/mnt/c/...`, que es mucho más lenta. Si la carpeta ya tiene el Nexus-Office de Windows (`venv\python.exe`), `install.sh` no lo borra: te pide copiarla a `~/`. Si descomprimes el zip desde Windows y se pierden los permisos, usa `bash install.sh`.

Aceleración incluida en los instaladores

Cada instalador lleva todo lo necesario para acelerar con la GPU o la NPU de su plataforma, sin internet. Nexus-Office elige al arrancar: prueba primero la variante de GPU adecuada a tu hardware y, si el equipo no tiene el driver (por ejemplo, `libcuda` o `vulkan-1.dll`), pasa a la siguiente y, al final, a la CPU.

Instalador	llama-server (chat, visión, embeddings)	sd-cli (imagen)	Wheels de aceleración
Windows x64	CUDA 12.9 (NVIDIA), Vulkan (NVIDIA, AMD, Intel), CPU	Vulkan, CPU	OpenVINO (Intel CPU/GPU/NPU), onnxruntime-qnn (NPU Qualcomm), PyTorch con DirectML (cualquier GPU)

Instalador	llama-server (chat, visión, embeddings)	sd-cli (imagen)	Wheels de aceleración
Windows ARM64	Vulkan (Adreno y otras), CPU	Vulkan, CPU	Las de Windows x64 (Python x64 emulado)
Linux x86_64	CUDA 12.9 (NVIDIA), Vulkan (NVIDIA, AMD, Intel), CPU	Vulkan, CPU	OpenVINO (Intel CPU/GPU/NPU)
Linux ARM64	CUDA (NVIDIA Jetson Orin, JetPack 6), Vulkan (Mali, Adreno, Raspberry Pi 5...), CPU	Vulkan, CPU	OpenVINO
macOS Apple Silicon	Metal (GPU de Apple), CPU	CPU	PyTorch con Metal (MPS), onnxruntime con Core ML
macOS Intel	CPU	CPU	—

- **Vulkan** acelera las GPU de cualquier fabricante con su driver normal. Lo compila `packaging/build_desktop_runtimes.sh` con zig desde las fuentes de PyPI; enlaza con el cargador de Vulkan del sistema (`libvulkan.so.1` en Linux, `vulkan-1.dll` en Windows), que instala cualquier driver de GPU.
- **CUDA** y **Metal** salen de los paquetes de llama.cpp de conda-forge (`packaging/build_gpu_runtimes.py`), con sus bibliotecas (cuBLAS en CUDA). Con NVIDIA hace falta el driver 525 o posterior.
- **No incluidos**: ROCm (AMD) y SYCL (Intel), que pesan varios GB; esas GPU van por Vulkan, y con internet se puede instalar la variante oficial desde *Sistema → Runtimes*. PyTorch con CUDA o ROCm (2-4 GB) tampoco: la música neuronal en esas GPU se instala con `nexus setup --all` y conexión. Sin PyTorch, Nexus-Office compone la música con el compositor procedural.
- **Selector de inicio (2.0)**: cada opción del selector de motor y CPU/GPU/NPU lleva su runtime dentro del paquete: llama-server en sus variantes, el motor integrado (llama-cpp-python) y llama-server con OpenVINO para la NPU Intel en Windows x64 y Linux x86_64. Ver ACELERADOR.md.
- `nexus setup` instala solo los extras que usa tu hardware. PyTorch se instala sin preguntar si sus wheels vienen en el instalador y el equipo tiene la GPU adecuada.

Limpieza al final de la instalación

Al terminar, `install.sh` / `install.bat` preguntan si quieres quitar lo que tu equipo no usa: las wheels que no se han instalado y las variantes de runtime de otras GPU. Después preguntan si guardas lo que sí se ha usado (las wheels instaladas y el Python de `runtime/`) para reinstalar sin internet con `--force --offline`. El runtime que usa tu equipo y su variante CPU de respaldo nunca se borran. El instalador de Windows limpia sin preguntar (`-Clean`) y el `.deb` no limpia.

```
./install.sh --clean          # sin preguntar: borra lo no usado y guarda lo usado (Windows: install.bat -Clean)
./install.sh --no-clean       # no limpia (Windows: install.bat -NoClean)
venv/bin/python3 -m nexus cleanup --dry-run   # en cualquier momento: muestra lo que se borraría
venv/bin/python3 -m nexus cleanup --no-keep   # borra también wheelhouse/ y runtime/ (Nexus-Office sigue
funcionando)
```

Sin terminal (instalación desatendida), no se limpia nada salvo con `--clean` o `NEXUS_CLEAN=yes`. Tras limpiar, la carpeta ya no sirve para instalar en otro equipo con otro hardware: para eso, guarda el instalador original.

AppImage y Flatpak

`packaging/build_linux_bundles.py` (lo llama `build.py` para cada Linux) genera `Nexus-Office-<ver>-linux-<arch>.AppImage` y `.flatpak` para x86_64 y aarch64. Los dos son de solo lectura, así que Nexus-Office va **ya instalado** dentro: Python con las dependencias base en su `site-packages` (instaladas con el pip del equipo de compilación y `--platform`, lo que permite construir el de ARM64 desde x86_64), más `bundled/runtimes`. Los datos van a `~/.local/share/nexus-office` (AppImage) o `~/.var/app/ai.nexus.NexusOffice` (Flatpak). Sin argumentos arrancan Nexus-Office; con argumentos son `python -m nexus …` (`servicio`, `modelos`, `oficina`, `chat`).

- **AppImage**: `appimagetool` y el `runtime-<arch>` de AppImage/type2-runtime (se descargan a `.cache/appimage/`). El servicio apunta al fichero `.AppImage` (variable `APPIMAGE`), no a la carpeta temporal donde se monta.
- **Flatpak**: se arma sin `flatpak-builder` (`flatpak build-export` + `build-bundle --runtime-repo` de Flathub): al instalarlo, Flatpak descarga `org.freedesktop.Platform 24.08`. Permisos: red, X11/Wayland, audio, todos los dispositivos (GPU) y `xdg-config/systemd` para dejar la unidad del servicio, que se activa desde fuera del sandbox con `systemctl --user enable --now nexus-office`.
- Los extras que se instalan con pip al vuelo (Sistema → extras) no están disponibles en estos dos formatos, porque la carpeta es de solo lectura: usa el `.run` o el `.deb` si los necesitas.

Servicio y descargas para la oficina

- **Servicio**: `python -m nexus servicio instalar|quitar|estado [--red] [--puerto P]` (`nexus/service.py`). Usa systemd (del sistema como root, de usuario con `loginctl enable-linger` si no), cron `@reboot` sin systemd, launchd (LaunchDaemon o LaunchAgent), el Programador de tareas de Windows (al iniciar el equipo como SYSTEM si es administrador, al iniciar sesión si no; reinicio ante fallos, sin límite de tiempo y regla de firewall con `--red`) o termux-services con Termux:Boot. La app Nexus-Office autónomo arranca su servicio en primer plano al encender (`BootReceiver`). Todo en SERVICIO.md.
- **Descargas**: `nexus/downloads.py` lista los ficheros de `<datos>/descargas/` (por ejemplo, `data/descargas/windows/Nexus-Office-Setup-2.2.0-windows-x64.exe`) y los clasifica por plataforma según su nombre. Para repartir los instaladores en la oficina, copia la carpeta de una entrega (`windows/`, `linux/`, `macos/`, `android/`…) ahí; nada la llena automáticamente. *Sistema → Descargas* marca el del sistema anfitrión y el del dispositivo que mira la página. Se sirven en `/descargas/<ruta>` con la misma protección de clave de red que el resto.

Empaquetado

Todo se construye en Linux (también WSL); el `.exe` de Windows sale de `makensis`, que funciona en Linux.

1. Prepara los runtimes en `.cache/` (una vez, o al cambiar de versión):

```
python3 packaging/build_python_runtimes.py      # CPython 3.11 portable de conda-forge (Linux, Windows x64, macOS)
bash packaging/build_desktop_runtimes.sh        # llama-server y sd-cli (CPU y Vulkan) para todas las combinaciones, con zig
#   (requisitos: sudo apt install cmake ninja-build python3 curl g++ glslc libvulkan-dev spirv-headers)
python3 packaging/build_gpu_runtimes.py         # CUDA (Windows x64, Linux x64, Jetson) y Metal (Apple Silicon) de conda-forge
#   (requisitos: pip install macholib pyelftools pefile)
python3 -m nexus wheels --target all --dest .cache/wheelhouse-all   # opcional: wheels de todas las plataformas
```

Todo queda en `.cache/desktop-runtimes/<plataforma>/` (`cpython-*.tar.gz`, `llama/<variante>/`, `sd/<variante>/`). Si falta el Python de `.cache/`, `build.py` lo baja de python-build-standalone (GitHub); si falta `.cache/wheelhouse-all`, descarga las wheels de cada plataforma. Windows ARM64 lleva el Python x64 (conda-forge no tiene CPython 3.11 para ARM64 y Windows 11 ARM lo emula) con sus wheels, y llama-server y sd-cli nativos ARM64.

2. Compila las APK (opcional; ver Apps Android). Las dos escriben en `dist/`, que `build.py` vacía al empezar: muévelas antes a otra carpeta, por ejemplo `dist-apk/`.

3. Genera los instaladores:

```
sudo apt install nsis                                # makensis, para el .exe (o brew install makensis / choco install nsis)
python3 packaging/build.py                           # todas las plataformas
python3 packaging/build.py --targets windows-x64 linux-aarch64
python3 packaging/build.py --apk dist-apk/*.apk --docs Manual-de-usuario.pdf Documentacion-tecnica.pdf
python3 packaging/build.py --keep --targets macos-arm64   # añade una plataforma sin borrar dist/
```

Opción	Qué hace
`--targets`	`windows-x64`, `windows-arm64`, `linux-x86_64`, `linux-aarch64`, `macos-arm64`, `macos-x86_64` (por defecto, todas).
`--apk`	APK ya compiladas, que se copian a `dist/android/`.
`--docs`	Ficheros extra para la raíz de `dist/` (los manuales en PDF, por ejemplo).
`--keep`	No borra `dist/` al empezar.

Sin `makensis`, `build.py` sigue con las demás plataformas y avisa de que falta el `.exe`.

Resultado:

```
dist/
  windows/  Nexus-Office-Setup-2.2.0-windows-x64.exe, Nexus-Office-Setup-2.2.0-windows-arm64.exe
  linux/    Nexus-Office-2.2.0-linux-x86_64.run, Nexus-Office-2.2.0-linux-aarch64.run,
            nexus-office_2.2.0_amd64.deb, nexus-office_2.2.0_arm64.deb
  macos/    Nexus-Office-2.2.0-macos-arm64.tar.gz, Nexus-Office-2.2.0-macos-x86_64.tar.gz
  android/  Nexus-Office-2.2.0-android-autonomo.apk, Nexus-Office-2.2.0-android-vpn.apk   (de --apk)
  Nexus-Office-2.2.0-codigo-fuente.zip, LEEME.txt, SHA256SUMS.txt (+ los ficheros de --docs)
```

Cada instalador lleva la carpeta portable completa: código, `runtime/` (CPython de la plataforma), `wheelhouse/` (dependencias y extras de voz, NPU y GPU, solo binarias) y `bundled/runtimes/` (llama-server y sd-cli para CPU y GPU). Ya no se generan los paquetes *online* que descargaban Python y las dependencias al instalar, ni los `.zip` por plataforma, ni el zip maestro, ni el paquete de Termux.

`Nexus-Office-2.2.0-codigo-fuente.zip` contiene solo código, tests, documentación, empaquetado y las apps Android, sin `venv/`, datos, modelos, cachés ni compilados; `build.py` se para si encuentra ficheros sensibles (claves, `.env`, `config.yaml`, registros). Las versiones de referencia de Python portable, llama.cpp, stable-diffusion.cpp y los modelos por perfil están en `requirements.txt`; se regenera con `packaging/requirements_ref.py`.

Los modelos no se incluyen en los instaladores (son de varios GB y tienen sus propias licencias): los descarga el propio instalador si hay internet, la aplicación desde *Modelos → Recomendados* o `nexus modelos --descargar`.

Para enviar los instaladores a un móvil o a otro canal con límite de tamaño (unos 30 MB por fichero), usa `venv/python.exe packaging/split_parts.py [--size 25] [ficheros]`. Deja el resultado en `dist/envio/`:

- Los ficheros pequeños se copian enteros.
- Los grandes se dividen en partes `.001`, `.002`… e incluyen `UNIR-<nombre>.bat` (Windows) y `unir-<nombre>.sh` (Linux, macOS o Termux) para reconstruirlos. Estos scripts comprueban la huella SHA-256 del fichero unido.
- `LEEME-DESCARGAS.txt` resume los ficheros con sus tamaños y huellas.

En la red de la oficina es más cómodo copiar `dist/` en `data/descargas/` del PC con Nexus-Office y descargarlos desde *Sistema → Descargas*.

Apps Android (versión 2.2.0)

Desde la 2.2 la app **autónoma** es la única que hace falta: lleva la IA en el móvil y, en *RED → Conectar a mi ordenador*, el mismo cliente de la app VPN (red local, HTTPS anclado y VPN WireGuard). La app VPN se sigue publicando para móviles que no pueden con la IA local. Las dos pueden instalarse a la vez:

	Nexus-Office VPN	**Nexus-Office autónomo**
Fichero	`Nexus-Office-2.2.0-android-vpn.apk`	`Nexus-Office-2.2.0-android-autonomo.apk`
Paquete	`ai.nexus.office.client` (sustituye a la app 1.3)	`ai.nexus.office.autonomo`
Android	7.0 o superior (API 24)	9.0 o superior (API 28), ARM64
Dónde corre la IA	En tu PC con Windows, Linux o macOS	En el propio móvil (modelo elegido según su RAM), en la GPU si llama.cpp puede usarla y va más rápida, si no en la CPU
Conexión	VPN WireGuard cifrada, HTTPS anclado o Wi-Fi	Su IA en `127.0.0.1` y, si quieres, tu PC por VPN WireGuard, HTTPS anclado o Wi-Fi
Arranque al encender	—	Opcional: *Sistema → Arranque automático* → **ARRANCAR AL ENCENDER**
Compilar	`venv/python packaging/build_apk.py`	`bash packaging/build_android_runtime.sh` y `venv/python packaging/build_apk_autonomo.py`

La versión de las dos APK sale de `nexus/__init__.py` más `APK_REVISION` (`packaging/build_apk.py`): la 2.2.0 revisión 0 da versionCode 10700. Desde la 2.2.0 los identificadores son `ai.nexus.office.client` y `ai.nexus.office.autonomo`, así que Nexus-Office no se instala encima de NexusAI: conviven.

Actualizar desde la app 1.3: la app Nexus-Office VPN 1.4 está firmada con una clave nueva. Desinstala antes la app Nexus-Office anterior, porque Android no deja instalar encima una app con otra firma. Después vuelve a conectar con el código de tu PC.

Compilar la app Nexus-Office VPN

Sirve en Linux, WSL o macOS.

1. Con el SDK de Android: copia `build-tools/34.0.0` a `.cache/android/build-tools` y `platforms/android-34/android.jar` a `.cache/android/platform/`. Necesitas también un JDK 17.
2. En Ubuntu o WSL, sin SDK: `bash sudo apt install aapt zipalign apksigner dalvik-exchange android-sdk-platform-23 openjdk-17-jdk-headless` Además necesitas el `android.jar` de la API 34 en `.cache/android/platform/` o en la variable `NEXUS_ANDROID_JAR`. Sirve el `android-all` de Robolectric de Maven Central.

Después:

```
venv/bin/python packaging/build_apk.py      # deja la APK en dist/Nexus-Office-<versión>-android-vpn.apk
```

`libwg-go.so` (WireGuard oficial, Apache 2.0) se descarga de Maven Central y se comprueba su huella SHA-256.

Para probar el túnel sin móvil, `sudo bash packaging/test_vpn_e2e.sh` monta dos espacios de red Linux:

- en uno, el PC levanta el túnel con la conf de Nexus-Office;
- en el otro, WireGuard se configura con el texto que genera la app.

La prueba comprueba que Nexus-Office responde por la VPN.

Compilar la app Nexus-Office autónomo

```
ANDROID_NDK_HOME=~/android-ndk bash packaging/build_android_runtime.sh   # Python y llama.cpp de CPU con zig; con
NDK, también el de GPU
venv/bin/python packaging/build_apk_autonomo.py    # deja la APK en dist/Nexus-Office-<versión>-android-autonomo.apk
(sin Gradle)
```

Detalles en android-autonomo/README.md.

Portabilidad

`venv/` contiene un **CPython completo** (python-build-standalone o conda-forge), no un entorno virtual que dependa del Python del sistema. Copiar la carpeta a otro equipo con el mismo sistema y arquitectura conserva modelos, memoria y configuración; Nexus-Office re-detecta el hardware al arrancar y adapta el perfil. Si tenías el servicio instalado, vuelve a instalarlo en la carpeta nueva.

Acceso desde otros dispositivos (móvil, tablet, consola, TV)

En *Red COOP → Conectar dispositivos* (modo admin) activa **Permitir acceso desde la red** y reinicia Nexus-Office (o instala el servicio con `--red`). Los dispositivos que no sean el propio PC necesitan la **clave de red** (se muestra ahí; la app Android y la URL con `?key=` la recuerdan). `/api/ping` es lo único público, para que la app encuentre el servidor.

Motor de IA y acelerador (CPU, GPU o NPU)

Desde la versión 2.0, al arrancar Nexus-Office puedes elegir a mano **con qué motor** se ejecuta el modelo de IA y **en qué chip**: el procesador (CPU), la tarjeta gráfica (GPU) o la unidad de IA (NPU). Si no eliges nada, lo decide Nexus-Office según tu equipo, como hasta ahora.

Dónde se elige

- **Al arrancar** aparece la ventana **MOTOR Y ACELERADOR**. Muestra tu procesador, memoria, GPU y NPU detectadas, y qué opciones están disponibles en este equipo (las que no, dicen por qué).
- **PROBAR** genera unas palabras con la opción marcada y dice cuánto ha tardado, los tokens por segundo y **dónde ha quedado el modelo de verdad** (por ejemplo, «GPU · 37/37 capas · Vulkan0 (NVIDIA GeForce RTX 4060)»).
- **ARRANCAR** la aplica. Con **Recordar y no volver a preguntar al abrir** (marcado por defecto) no vuelve a preguntar.
- Para cambiarla después: *Sistema → MOTOR Y CPU/GPU/NPU* (junto a *Redetectar*).
- Desde la terminal: `python -m nexus run --motor llamaserver --acelerador gpu` (cualquiera de las dos opciones fija la elección y no pregunta).

Perfiles de hardware (2.2)

Un **perfil** guarda de una vez cómo usar tu equipo: motor, acelerador, GPU, modelo, contexto, capas en la GPU y dónde va la memoria de la conversación (caché KV: en la **VRAM**, más rápida, o en la **RAM**, para que quepa un modelo más grande en una gráfica pequeña), y si se bloquea el modelo en RAM (`--mlock`).

Perfil de serie	Para qué
Automático	Lo decide Nexus-Office según el equipo (como siempre)
Rendimiento	GPU, modelo entero en la VRAM y contexto máximo
Equilibrado	GPU si la hay, contexto moderado (8192)
Modelo grande	Parte del modelo en la VRAM y el resto y el contexto en la RAM (bloqueada)
Ahorro	CPU, contexto corto (2048) y la mitad de los hilos: menos consumo
Solo CPU	Sin GPU ni NPU

Se elige en el menú **PERFIL** de la barra superior, que muestra solo lo que tu hardware permite. **Editar perfiles** crea, copia o cambia los tuyos (si cambias uno de serie y lo borras, vuelve a ser el original). Al activar uno, Nexus-Office reinicia el motor con esos valores. API: `GET /api/hw/perfiles`, `PUT` y `DELETE /api/hw/perfiles/{id}`, `POST /api/hw/perfiles/{id}/activar`. Configuración: `hardware.perfil` y `hardware.perfiles`.

Equipos con dos o más GPU y Linux (2.2)

- llama-server se arranca con `--device` para la GPU elegida (la lista sale de `llama-server --list-devices`). Antes podía caer en la integrada aunque hubiera una dedicada, o repartir capas entre las dos.
- En el menú **IA** eliges la GPU cuando hay más de una (`POST /api/hw/gpu`, `hardware.gpu`). En *Automático* se prefiere la dedicada con más VRAM.
- Los dispositivos **de software** (llvmpipe, lavapipe, SwiftShader) no se usan nunca: en Linux sin driver de Vulkan aparecían como «GPU» e iban más lentos que la CPU. Si es lo único que hay, se queda en CPU y lo dice.
- El nombre de la GPU se reconoce aunque llama.cpp le añada sufijos (por ejemplo «Intel(R) UHD Graphics 770 (ADL-S GT1)»).
- Si la GPU elegida falla al arrancar, Nexus-Office vuelve a la CPU y lo muestra en el menú **IA** (`gpu_error`).

No se ha podido probar con dos GPU físicas; la elección se ha probado con la salida real de `--list-devices` de varios equipos.

Motores

Motor	Qué es	CPU	GPU	NPU	Visión
Automático	El mejor disponible (normalmente llama-server)	✓	✓	✓	✓
llama-server	llama.cpp oficial como proceso aparte	✓	CUDA, Vulkan, Metal	OpenVINO (Intel), Hexagon (Android)	✓
llama.cpp integrado	llama-cpp-python dentro de Nexus-Office	✓	Metal en Mac	—	—

Los dos motores van **incluidos en los instaladores offline**, con su runtime: no se descarga nada al elegirlos. El integrado se instala desde el propio paquete la primera vez que lo eliges (unos segundos, sin internet).

Ollama ya no aparece en el selector porque es otra aplicación que hay que instalar aparte y no viene en el paquete. Si lo tienes instalado, Nexus-Office lo sigue usando cuando no hay otro motor.

Aceleradores

- **CPU**: siempre disponible. El modelo no usa la GPU aunque la haya.
- **GPU**: NVIDIA con CUDA; AMD, Intel, NVIDIA, Adreno, Mali o Raspberry Pi 5 con Vulkan; Mac con Metal; en Android, la GPU del móvil (OpenCL o Vulkan). Con una GPU integrada se avisa de que puede ir más lenta que la CPU con modelos grandes.
- **NPU**:
- **Intel Core Ultra (AI Boost)**, Windows x64 y Linux x86_64: llama-server con **OpenVINO** en la NPU (`GGML_OPENVINO_DEVICE=NPU`). La NPU trabaja con grafos fijos, así que se usa una sola conversación a la vez y un contexto de hasta 4096 tokens.
- **Snapdragon con Hexagon v73 o posterior** (app Android autónoma): llama.cpp en la NPU (HTP). Si el driver del móvil no lo permite, pasa solo a la GPU y luego a la CPU.
- **AMD Ryzen AI, Apple Neural Engine y Qualcomm en Windows**: llama.cpp no puede ejecutar ahí el modelo de texto. El selector lo dice y usa la GPU (o la CPU). Esas NPU se siguen usando para embeddings y voz cuando su runtime está instalado.

Cómo se comprueba que acelera de verdad

Tras arrancar llama-server, Nexus-Office lee su registro (`logs/llama-server.log`) y anota el dispositivo real: capas cargadas en la GPU, dispositivo CUDA, Vulkan o Metal, la NPU Hexagon u OpenVINO. Si lo pedido no se ha cumplido (por ejemplo, falta el driver de la GPU), lo dice en el selector y en el registro, y el modelo sigue en lo que sí funciona.

Runtimes incluidos por instalador

Instalador	llama-server	Motor integrado	NPU
Windows x64	CPU, Vulkan, CUDA 12	llama-cpp-python (CPU)	OpenVINO (Intel AI Boost)
Windows ARM64	CPU, Vulkan	llama-cpp-python x64 (emulado)	—
Linux x86_64 (.deb, .run, AppImage, Flatpak)	CPU, Vulkan, CUDA 12	llama-cpp-python (CPU)	OpenVINO (Intel AI Boost)
Linux ARM64 y Raspberry Pi	CPU, Vulkan, CUDA (Jetson)	llama-cpp-python (CPU, NEON)	—
macOS Apple Silicon	CPU, Metal	llama-cpp-python (Metal)	—
macOS Intel	CPU	llama-cpp-python (CPU)	—
Android autónoma	CPU, OpenCL/Vulkan	—	Hexagon HTP

Los runtimes del selector (llama-cpp-python y llama-server con OpenVINO) se compilan y prueban con `.github/workflows/selector-runtimes.yml` y la construcción de los instaladores los toma de ahí.

Configuración

- `llm.engine`: `auto`, `llamaserver` o `llamacpp`.
- `hardware.accel`: `auto`, `cpu`, `gpu` o `npu`.
- `hardware.llama_variant`: variante de llama-server elegida (`cpu`, `vulkan`, `cuda`, `metal`, `openvino`).
- `startup.ask_accel`: `true` para preguntar al arrancar.

API: `GET /api/accel` (opciones, elección y dispositivo efectivo), `POST /api/accel {engine, accel, remember}`, `POST /api/accel/test` (genera 16 tokens y mide).

Motores de inferencia por hardware

Desde la 4.0.0 Agente IA (edición Python) elige el **motor de inferencia** que mejor acelera el chip del equipo donde se ejecuta, en lugar de usar siempre llama.cpp. Cada fabricante tiene su motor optimizado: los kernels de TensorRT-LLM para NVIDIA, OpenVINO para la NPU de Intel, Genie o ONNX Runtime con QNN para la NPU Hexagon de Qualcomm, LiteRT-LM para la TPU de los Pixel... llama.cpp sigue de respaldo en todos los equipos.

Motores

Motor	Aceleradores	Formato de modelo	Va en el paquete offline
ONNX Runtime GenAI	CUDA, DirectML (cualquier GPU en Windows), QNN (NPU Snapdragon X), OpenVINO, VitisAI (NPU Ryzen AI), CPU	carpeta con `genai_config.json`	Windows x64 (DirectML), Windows ARM64, Linux, macOS
OpenVINO GenAI	NPU Intel Core Ultra, iGPU Xe y Arc, CPU	carpeta con `openvino_model.xml`	Windows x64, Linux x86_64, macOS
LiteRT-LM	NPU de Tensor, Snapdragon, MediaTek e Intel; GPU; CPU (XNNPACK)	`.litertlm`	Windows x64 (con NPU Intel), Linux, macOS Apple Silicon (rueda `litert-lm-api` y `motores-npu.yml`)
ExecuTorch	XNNPACK (CPU), QNN, Vulkan, CoreML, MPS (según el `.pte`)	`.pte` + tokenizer	Windows x64 y macOS Apple Silicon; en Linux aparte (arrastra torch con CUDA, unos 3 GB): `nexus setup --all` con internet
MLC LLM	CUDA, ROCm, Vulkan, Metal, OpenCL (Adreno, Mali)	carpeta con `mlc-chat-config.json`	aparte (no hay rueda en PyPI): `nexus setup --all` con internet
TensorRT-LLM	NVIDIA (Tensor Cores)	checkpoint de Hugging Face o `rank0.engine`	aparte (varios GB): `nexus setup --all` con internet
Qualcomm Genie	NPU Hexagon (HTP)	carpeta de Qualcomm AI Hub con `genie_config.json`	Windows ARM64 y Linux ARM64 (QAIRT, de `motores-npu.yml`)
llama.cpp	CPU, CUDA, ROCm, Vulkan, Metal, SYCL	`.gguf`	siempre (respaldo)

Si falta el paquete o la biblioteca de un motor, Agente IA dice por qué y usa el siguiente.

Android (APK autónoma), desde la 4.0.2

El Python de la app usa su propio cargador (musl) y no puede cargar con ctypes las bibliotecas de Android (bionic). Por eso en el móvil cada motor corre fuera del proceso de Python (`nexus/motores/auxiliar.py`):

Motor	Cómo va en el móvil	Aceleradores
Qualcomm Genie (QNN)	proceso nativo `libnexusmotor.so motor genie` (el ejecutable Android de la edición C++)	NPU Hexagon v68 a v81 (Snapdragon 8 Gen 1 a 8 Elite Gen 5)
LiteRT-LM	proceso nativo `libnexusmotor.so motor litert`, con el dispatch de NPU de Qualcomm, MediaTek y Google Tensor	NPU, GPU (OpenCL/WebGPU), CPU
ONNX Runtime GenAI	proceso nativo `libnexusmotor.so motor onnx`, ONNX Runtime con el proveedor QNN	NPU (QNN), CPU
ExecuTorch	puente Java de la app (`MotorExecuTorch.java`, `LlmModule`) en 127.0.0.1 con clave	CPU (XNNPACK), NPU (delegado QNN, si el `.pte` lo lleva)
llama.cpp	`llama-server` (como antes)	CPU, GPU (OpenCL/Vulkan), NPU Hexagon
TensorRT-LLM, OpenVINO GenAI, MLC LLM	**no aplican** en Android: se muestran aparte en *Sistema → Motores* con el motivo	—

Todas las bibliotecas van en `lib/arm64-v8a` de la APK (Android solo deja ejecutar código nativo desde ahí) y las recoge `apk.yml`: LiteRT-LM y QAIRT 2.42 (Genie, QNN HTP) de `motores-npu.yml`, ONNX Runtime 1.29 con QNN y ONNX Runtime GenAI 0.19 de Maven y NuGet, ExecuTorch 1.5.1 con QNN de Maven, y el proceso auxiliar de la Release v3.2.3-cpp. La NPU busca sus programas (skel) en `ADSP_LIBRARY_PATH`, que apunta a esa carpeta.

Modelos para la NPU del móvil: los `.litertlm` del catálogo se descargan en la versión compilada para el SoC del móvil si el repositorio la tiene (`..._sm8750.litertlm` en un Snapdragon 8 Elite); si no, la general (GPU y CPU). LiteRT-LM solo

prueba la NPU con modelos compilados para un SoC. Genie usa carpetas de Qualcomm AI Hub (`genie_config.json` y los `.bin` del Snapdragon concreto): se importan desde *Modelos → Importar*.

Qué motor va con cada chip

Agente IA resume el hardware en una familia y parte de este orden (ver `nexus/motores/chips.py`):

Chip	Orden de partida
Snapdragon (móvil y X Elite/Plus)	Genie (NPU) → ONNX Runtime QNN → LiteRT-LM NPU → ExecuTorch QNN → MLC (Adreno) → llama.cpp
Google Tensor	LiteRT-LM NPU → LiteRT-LM GPU → MLC → ExecuTorch → llama.cpp
MediaTek, Exynos	LiteRT-LM NPU → LiteRT-LM GPU → MLC → ExecuTorch → llama.cpp
NVIDIA	TensorRT-LLM → ONNX Runtime CUDA (→ DirectML en Windows) → MLC CUDA → llama.cpp CUDA
AMD	(Windows: NPU Ryzen AI con VitisAI) → ONNX Runtime DirectML/ROCm → MLC → ExecuTorch Vulkan → llama.cpp
Intel	OpenVINO NPU → OpenVINO GPU → ONNX Runtime OpenVINO → DirectML/MLC → OpenVINO CPU → llama.cpp
Apple Silicon	MLC Metal → llama.cpp Metal → ExecuTorch MPS
Solo CPU	llama.cpp → OpenVINO CPU (x86) o LiteRT-LM (ARM) → ONNX Runtime CPU → ExecuTorch

El banco manda sobre la tabla

La tabla es solo el punto de partida. En **Sistema → Motores de inferencia → MEDIR TODOS LOS MOTORES** (o `nexus motores banco` en la terminal) Agente IA genera una respuesta corta con cada modelo de texto instalado en su motor y mide los **tokens por segundo** y el **tiempo hasta el primer token**. Desde ese momento se elige el más rápido medido en tu equipo, aunque la tabla diga otra cosa (por ejemplo, si en tu portátil llama.cpp en la iGPU supera a OpenVINO en la NPU). El resultado queda en `data/motores/banco.json`.

Entre los modelos medidos que van cómodos (8 tokens/s o más) se elige **el mayor**, que suele responder mejor, y a igual tamaño el motor más rápido. Si ninguno llega a esa velocidad, el más rápido. Antes de medir se pide una respuesta mínima para que la carga del modelo (que se apunta aparte) no cuente en el tiempo hasta el primer token. Al cambiar de motor se descarga el modelo del anterior para no tener dos en memoria.

```
python -m nexus motores          # chip, orden, motores instalados y modelos con su motor
python -m nexus motores banco    # mide todos y muestra la tabla de tokens/s
```

Aceleradores y caídas

Cada motor prueba sus aceleradores en orden (por ejemplo NPU → GPU → CPU). Si uno no arranca con un modelo, se recuerda y se usa el siguiente. Mientras un motor carga o genera, Agente IA deja una marca en disco: si el proceso se cae (un driver de NPU o GPU que falla), al volver a arrancar esa combinación queda apuntada como **caída** y no se usa hasta pulsar *OLVIDAR CAÍDAS*.

Se puede fijar el motor y el acelerador en el mismo panel, o en `config.yaml`:

```
llm:
  motor: auto            # auto | onnx | openvino | litert | executorch | mlc | tensorrt | genie | llamaserver
  motor_acelerador: ""   # "" = automático; p. ej. NPU, GPU, CPU (OpenVINO), cuda, dml, qnn (ONNX), npu (LiteRT)
```

Modelos para cada motor

En **Modelos** aparecen, además de los GGUF, los recomendados para los motores de tu chip:

Modelo	Motor	Repositorio
Qwen2.5 1.5B int4	OpenVINO	`OpenVINO/Qwen2.5-1.5B-Instruct-int4-ov`
Phi-3.5 mini int4	OpenVINO	`OpenVINO/Phi-3.5-mini-instruct-int4-ov`
Qwen3 8B int4	OpenVINO	`OpenVINO/Qwen3-8B-int4-ov`
Phi-4 mini (CPU/QNN)	ONNX Runtime GenAI	`microsoft/Phi-4-mini-instruct-onnx` (`cpu_and_mobile/...`)

Modelo	Motor	Repositorio
Phi-4 mini (GPU)	ONNX Runtime GenAI	`microsoft/Phi-4-mini-instruct-onnx` (`gpu/...`)
Qwen2.5 1.5B / Gemma 4 E2B	LiteRT-LM	`litert-community/...`
Qwen2.5 1.5B / Qwen3 4B q4f16	MLC LLM	`mlc-ai/...-MLC`
Qwen3 4B 8da4w	ExecuTorch	`pytorch/Qwen3-4B-INT8-INT4`
Qwen2.5 1.5B	TensorRT-LLM	`Qwen/Qwen2.5-1.5B-Instruct` (se compila para tu GPU la primera vez)

Los modelos de Genie salen de Qualcomm AI Hub compilados para cada Snapdragon: se importan como carpeta en *Modelos → Importar*. También se puede importar cualquier carpeta o fichero de estos formatos: Agente IA reconoce el formato y lo asigna a su motor.

Pruebas

- `tests/test_motores.py`: formatos, matriz por chip, selector, banco, caídas y backend con un motor simulado.
- `.github/workflows/motores-python.yml`: descarga modelos reales y mide llama.cpp, ONNX Runtime GenAI y OpenVINO GenAI en los runners de GitHub (CPU x86_64 y ARM64); la tabla queda en el resumen de la ejecución.
- Primera medida en los runners (2026-11-10, solo CPU, respuesta corta):

Runner	Modelo	Motor	tok/s	1.er token
x86_64	Qwen2.5 1.5B	llama.cpp (Q4_K_M)	24.7	4.50 s
x86_64	Qwen2.5 1.5B	OpenVINO (int4)	15.3	7.41 s
x86_64	Phi-4 mini	ONNX Runtime GenAI (int4)	12.0	6.65 s
x86_64	Phi-4 mini	llama.cpp (Q4_K_M)	10.5	4.04 s
x86_64	Phi-3.5 mini	OpenVINO (int4)	5.2	17.68 s
ARM64	Qwen2.5 1.5B	llama.cpp	31.5	2.21 s
ARM64	Phi-4 mini	llama.cpp	12.8	3.82 s
ARM64	Phi-4 mini	ONNX Runtime GenAI	12.1	5.05 s

En CPU pura llama.cpp sigue siendo igual o más rápido salvo Phi-4 mini en x86 con ONNX Runtime: la ventaja de los otros motores está en la NPU y la GPU, y por eso manda el banco de cada equipo. - Las NPU y GPU no se pueden probar en GitHub: se miden en el equipo de cada usuario con el banco.

Nexus-Office Autónomo para Android

APK: `Nexus-Office-<versión>-android-autonomo.apk`. Paquete `ai.nexus.office.autonomo`, para **Android 9 o superior con ARM64**, que son prácticamente todos los móviles desde 2017.

Nexus-Office completo dentro del móvil, sin PC. **La APK lo lleva todo menos los modelos de IA**:

Dentro de la APK	Qué es
Python 3.12 para ARM64	Con ssl, sqlite3, ctypes, lzma y bz2
Dependencias de Nexus-Office	FastAPI, uvicorn, numpy, pydantic, cryptography, Pillow, psutil…
El código de Nexus-Office	Memoria, RAG, importación de documentos y ZIP/TAR, agentes, rastreo web y webui
`llama-server` (llama.cpp) en tres variantes de CPU	ARMv8 básica, ARMv8.2 con *dotprod* y ARMv8.6 con *i8mm*. Nexus-Office usa la más rápida que admite la CPU.
`llama-server` con GPU	llama.cpp enlazado con la libc de Android, con los backends **OpenCL** (Adreno de Qualcomm) y **Vulkan** (Mali e Immortalis de ARM, Xclipse de Samsung, PowerVR…) y la CPU en siete variantes de ARM (hasta ARMv9.2 con SVE2 y SME). Cada backend se carga solo si el móvil tiene su driver.

Modelos: al abrir la app por primera vez, Nexus-Office elige el modelo adecuado a la RAM del móvil y solo descarga ese.

RAM del móvil	Modelo que descarga
Menos de 10 GB	Qwen3-VL 2B Thinking (≈ 1,5 GB)
10 GB o más	Qwen3-VL 4B Thinking (≈ 2,8 GB)

Si el modelo va lento en ese móvil, Nexus-Office propone cambiar al más pequeño (y al revés si sobra potencia). En **Sistema > Arranque automático** se puede activar que Nexus-Office arranque solo al encender el móvil.

Con Wi-Fi empieza a descargar solo. Con datos móviles, pregunta antes. En *Modelos* se puede cambiar.

Oficina para pymes: los perfiles por gremio (herrería, calderería, reformas…) también van en la APK. Se eligen en *Agentes → Perfil de oficina* (modo ADMIN) y funcionan sin internet. Ver Oficina para pymes.

CPU, GPU y NPU

Al arrancar, Nexus-Office mira qué tiene el móvil (`nexus/hardware/android_accel.py`) y lo muestra en *Sistema → Hardware*:

Pieza	Cómo la detecta	Para qué la usa
CPU	El SoC (`Build.SOC_MODEL`, `ro.soc.model`, `ro.board.platform`) con su nombre comercial (p. ej. «Qualcomm SM8650 (Snapdragon 8 Gen 3)») y sus núcleos rápidos	La variante de llama.cpp y los hilos
GPU	El nombre por EGL (lo escribe la app en `device_info.json`) y, sobre todo, `libllama-server-gpu.so --list-devices`, que dice qué GPU puede usar llama.cpp de verdad	El modelo entero en la GPU: **Adreno por OpenCL**, el resto **por Vulkan**. En Adreno no se usa Vulkan porque en llama.cpp va más lento que la CPU.
NPU	Por el SoC y por las bibliotecas del fabricante: Hexagon (Qualcomm), APU (MediaTek), TPU (Google Tensor), NPU (Exynos). La Hexagon de los **Snapdragon 8 Gen 2 o posteriores** (8 Gen 3, 8 Elite, 8 Elite Gen 5, 8s Gen 3…) aparece además en `--list-devices` como `HTP0`	En esos Snapdragon, **el modelo entero en la NPU** con el backend Hexagon de llama.cpp, por delante de la GPU. Las demás NPU (MediaTek, Tensor, Exynos y Snapdragon anteriores) no tienen backend en llama.cpp: se informa de ellas y el modelo va en la GPU o en la CPU.

El orden es **NPU → GPU → CPU**. Si hay NPU o GPU utilizable, el perfil pone todas las capas en ella (`-ngl 999 --device HTP0`, `GPUOpenCL` o `Vulkan0`). Después el autoajuste **mide** unas respuestas con ella y otras con la CPU y se queda con lo más rápido, porque hay GPU de gama baja que van peor que los núcleos grandes. Si la NPU falla al arrancar llama-server, Nexus-Office lo apunta (`android_accel.json`, campo `npu_broken`) y pasa a la GPU; si falla la GPU (`gpu_broken`), sigue en la CPU, sin cortar la conversación. No vuelve a probar lo que falló hasta que se actualiza la app o Android.

Cómo usa la NPU: la APK lleva `libggml-hexagon.so` (la parte que corre en Android) y los programas del DSP `libggml-htp-v73/v75/v79/v81.so`, uno por generación de Hexagon. `libggml-hexagon.so` abre `libcdsprpc.so` (FastRPC, del fabricante, en `/vendor/lib64`) y carga en el DSP el programa de su generación, que busca en `ADSP_LIBRARY_PATH` (la

carpeta de bibliotecas de la app). En un móvil sin `libcdsprpc.so` el backend no encuentra la NPU y no pasa nada. Se compila con el Hexagon SDK de Qualcomm (`.github/workflows/android-npu.yml`). El backend Hexagon de llama.cpp aún es experimental, y la NPU no se ha podido probar en un móvil real al construir esta versión: si en el tuyo falla, la app sigue sola en la GPU o en la CPU.

La imagen (stable-diffusion.cpp) y la voz Piper no van en la APK: la voz es la del sistema.

Se optimiza sola

Nexus-Office se ajusta al móvil al arrancar y mientras lo usas (`nexus/android_tune.py`):

Qué ajusta	Cómo
NPU, GPU o CPU	Con NPU o GPU utilizable, compara la velocidad real con ella y con la CPU (3 respuestas cada una) y se queda con lo más rápido. Lo recuerda por modelo y versión. Si la NPU no arranca, compara la GPU.
Variante de llama.cpp	La más rápida que admite la CPU: i8mm, dotprod o ARMv8 básica
Hilos	Empieza con los núcleos rápidos. Después mide la velocidad real de tus respuestas con 2 o 3 valores y se queda con el mejor. Vuelve a medir al cambiar de modelo o de versión, y de vez en cuando.
Temperatura y batería	Móvil caliente, batería baja sin cargar o ahorro de energía: la mitad de hilos hasta que se normaliza
Contexto	4096 de inicio. Si falta RAM libre, lo reduce; si sobra en sesiones estables, lo amplía hasta 8192
Modelo	Si va lento propone uno más pequeño; si sobra velocidad y RAM, uno mayor. Lo descarga y lo activa al aceptar.

Los cambios se aplican entre respuestas. Para ver lo que ha decidido: *Sistema → Autoajuste*, o `GET /api/android/autoajuste`.

Cómo funciona

- **Actualizar**: instala la APK nueva encima de la anterior. Se conservan los datos y los modelos, y la app prepara el runtime nuevo.
- **Primer arranque**: la app comprueba que hay unos 350 MB libres. Después descomprime `assets/runtime.zip` en sus datos privados con varios hilos, mostrando el progreso, y verifica ficheros y bytes contra `assets/runtime.json`. Luego arranca en pocos segundos.
- **Sin cuelgues**: en cada arranque comprueba que el entorno sigue completo y, si no, lo repara solo. Si Nexus-Office no responde en 4 minutos, lo para y muestra las últimas líneas del registro. La pantalla de arranque ofrece **Reintentar**, **Reparar** y **Copiar registro**. El registro está en `nexus.log`, y el del arranque anterior en `nexus.prev.log`.
- **Tras cada inicio**: limpia temporales, optimiza la base de datos, aplica los hilos medidos y precarga el modelo si el móvil está en condiciones.
- **Servicio en primer plano**: mantiene Nexus-Office en `127.0.0.1:8775`, solo accesible desde el propio móvil. La notificación «Nexus-Office está activo» tiene el botón **Detener**.
- **Cómo se arranca Python**: Android no deja ejecutar ficheros de los datos de la app. La app ejecuta el cargador de musl (`libnexusld.so`), que va entre las bibliotecas nativas de la APK, y es el cargador quien carga el intérprete. Nexus-Office lanza `llama-server` directamente desde las bibliotecas nativas.
- **Por qué hay dos llama.cpp**: los de CPU son estáticos con musl, como Python, y no pueden cargar los drivers de la GPU, que son bibliotecas de Android. El de GPU está enlazado con la libc de Android y se lanza con `LD_LIBRARY_PATH` en sus bibliotecas, `/system/lib64` (Vulkan) y `/vendor/lib64` (el `libOpenCL.so` del fabricante).
- **Llamadas al sistema que Android prohíbe**: si un paquete usa una que el filtro seccomp de Android no permite (por ejemplo `mbind`, de OpenBLAS), Android mata el proceso con el «código 159». La app carga `libnexussys.so`, que convierte esas llamadas en «no disponible» (`ENOSYS`) para que Nexus-Office siga funcionando. Cada una queda anotada en `nexus.log`.
- **DNS**: el Python de la app no puede usar el DNS de Android. La app le pasa los servidores DNS de la red activa y Nexus-Office resuelve los nombres él mismo (`nexus/android_dns.py`). Si no responden, usa 1.1.1.1, 8.8.8.8 o 9.9.9.9.
- **Interfaz**: voz del sistema, adjuntar varios ficheros, y botones COPIAR y DESCARGAR.

Conectar a mi ordenador (desde 2.2)

La misma APK también se conecta al Nexus-Office de tu PC (Windows, Linux o macOS): en la barra superior, **RED → Conectar a mi ordenador**. Ofrece lo mismo que la app VPN:

- **Red local**: busca tu PC en la Wi-Fi o escribe su dirección.
- **Internet**: con el código de emparejamiento del PC, por HTTPS con el certificado anclado.
- **VPN cifrada (WireGuard)**: con el código NEXUSV1 del panel VPN del PC. Solo pasa por la VPN el tráfico de la app.

El botón **USAR NEXUS-OFFICE EN ESTE MÓVIL** vuelve a la IA del teléfono, que sigue funcionando sin conexión. La app VPN ligera (`ai.nexus.office.client`, carpeta `../android`) se sigue publicando para móviles que no son ARM64 o con poca memoria; las dos pueden estar instaladas a la vez.

Compilar

En Linux o WSL, sin Gradle. Para la parte de CPU no hace falta el SDK ni el NDK de Google; para el llama-server con GPU hace falta el **NDK** (r26 o posterior) en `ANDROID_NDK_HOME` o en `.cache/android/ndk/`, y las cabeceras `opencl-c-headers`, `libvulkan-dev` y `spirv-headers` de Ubuntu. Sin NDK la APK sale igual, solo con CPU.

```
sudo apt install python3.12 cmake ninja-build autoconf automake libtool tcl curl binutils-aarch64-linux-gnu \
                 qemu-user-static aapt zipalign apksigner dalvik-exchange android-sdk-platform-23 openjdk-17-jdk-
headless
bash packaging/build_android_runtime.sh --probar   # Python + llama.cpp para ARM64
venv/bin/python packaging/build_apk_autonomo.py    # deja dist/Nexus-Office-<versión>-android-autonomo.apk
```

`build_android_runtime.sh` lo hace todo desde el código fuente:

1. Compila Python y sus bibliotecas con **zig** (PyPI) para `aarch64-linux-musl`. Las fuentes salen de Ubuntu.
2. Instala las ruedas `musllinux_aarch64` de PyPI.
3. Compila `llama-server` estático desde el paquete fuente de `llama-cpp-python`.
4. Con el NDK, compila el mismo llama.cpp para Android con `GGML_BACKEND_DL`, Vulkan, OpenCL (kernels de Adreno) y todas las variantes de CPU de ARM. Para enlazar OpenCL genera un `libOpenCL.so` de solo nombres que **no** va en la APK: en el móvil se carga el del fabricante.
5. Con `--probar`, arranca Nexus-Office emulando ARM64 con qemu y comprueba que responde.

El resultado queda en `.cache/android-runtime/`. Solo hace falta repetirlo al cambiar dependencias.

`build_apk_autonomo.py` necesita el mismo `android.jar` que la app VPN (ver PLATAFORMAS) y firma con la misma clave.

La versión de la APK sale de `nexus/__init__.py` y de `APK_REVISION` en `packaging/build_apk.py`: 1.6.0 con la revisión 1 (GPU) da versionCode 10601, así que se instala encima de la 10600 conservando datos y modelos.

Notas

- **RAR y 7z**: no están dentro del móvil, porque no hay `unrar` ni `7z`. ZIP y TAR sí. Para importar un RAR, conviértelo a ZIP o impórtalo en el Nexus-Office del PC.
- **Paralelismo**: el rastreo y la importación usan hilos, porque en Android no se lanzan procesos hijo de Python.
- **Herramientas que ejecutan código Python**: se lanzan a través del mismo cargador (`nexus/proc.py: python_cmd`).
- **Datos**: se guardan en el almacenamiento privado de la app y se borran al desinstalarla.
- **x86_64**: emuladores y Chromebooks Intel no están incluidos. La APK es solo ARM64.

Guía de administrador / desarrollador

Pulsa **ADMIN**. La primera vez se crea un PIN (mínimo 4 caracteres, guardado como hash PBKDF2-SHA256 con sal). La sesión de administrador dura 8 horas o hasta volver a USUARIO.

Qué desbloquea

Vista	Funciones de administrador
Agentes	Crear, editar y borrar agentes: prompt, modelo (`default`, `ollama:…`, `openai:…`, `peer:NODO`), temperatura, tokens, herramientas permitidas, memoria, delegación, palabras clave, color e icono.
Salón	Aprobar o rechazar los **agentes propuestos** por el salón.
Modelos	Buscar en los 5 repositorios, listar ficheros, descargar cualquier modelo, importar ficheros/carpetas locales, usar modelos externos (Ollama, OpenAI-compatible, nodo de la red).
Herramientas	Política por riesgo, activar/desactivar herramientas, revisar/editar/probar/aprobar/rechazar propuestas de los agentes, compartirlas con la red, auditoría completa.
Red	Crear red, generar invitaciones, conexiones entrantes, qué se ofrece a la red, nodos y URLs públicas; **permitir acceso desde la red local** (móvil, tablet, consola) con clave de red.
Sistema	Instalar variantes de runtime, **arranque automático** (instalar o quitar el servicio), **actualizar drivers de la GPU** (abre el actualizador oficial), re-detectar hardware, historial de sesiones, registro, **apagado limpio**. El panel **Descargas** (instaladores para la red) lo ven todos.
Ajustes	Toda la configuración (`llm`, `memory`, `tools`, `hardware`, `models`, `media`, `coop`, `lounge`, `server`) y cambio de PIN.

Configuración destacada (`data/config.yaml`)

```
llm:
  backend_order: [llamaserver, llamacpp, ollama, openai, transformers, builtin]
  default_model: qwen3.5-2b               # id del catálogo o "ollama:modelo", "openai:modelo", "peer:nodo"
  openai_base_url: http://127.0.0.1:1234/v1   # LM Studio, vLLM, llama-server externo, OpenRouter...
  reasoning: auto           # auto | off (modelos Thinking: razonan antes de contestar)
  reasoning_budget: 2048    # tokens de razonamiento en el chat (se suman a max_tokens)
  reasoning_budget_tasks: 768   # en tareas internas (resúmenes, extracción)
memory:
  embed_model: qwen3-embedding-0.6b   # embeddings GGUF en CPU (ver MEMORIA.md)
  embed_dims: 0             # recorte Matryoshka para ahorrar RAM (p.ej. 512)
  organizer: {min_affinity: 0.35, margin: 0.03}   # que va al especialista y que al grafo general
  zip_reason_docs: 12       # documentos de un ZIP que razona el LLM
rag:
  crawl_batch_pages: 0      # páginas por lote al rastrear dominios (0 = según la RAM)
  allow_private_urls: false # permitir rastrear la red local / localhost
tools:
  policy: {low: auto, medium: ask, high: admin}   # auto | ask | admin | deny
  sandbox_community_tools: true
hardware:
  profile_override: ""      # cuda | rocm | vulkan | xpu | openvino | metal | cpu
  llama_variant: ""         # forzar variante del runtime llama.cpp
  adaptive_tuning: true
models:
  huggingface_token: ""     # modelos con acceso restringido
  civitai_token: ""
  kaggle_username: ""
  kaggle_key: ""
server: {host: 127.0.0.1, port: 8775}
```

Acceso remoto por internet

Red COOP → Acceso remoto: - Activar el puerto HTTPS (TLS 1.3). - Abrir el puerto en el router por UPnP. - Generar códigos de emparejamiento de un solo uso. - Revocar dispositivos.

Ajustes → Asistente del router: detecta el router, abre su configuración y da la regla exacta para abrir el puerto.

Guía completa: Acceso remoto.

Autoajuste

En **Sistema → Autoajuste adaptativo** verás qué cambió en cada arranque y por qué, y las pruebas de hilos (tokens/s por número de hilos). Desactívalo con `hardware.adaptive_tuning: false`. Usa **Apagar Nexus-Office** para que la sesión cuente como cierre limpio.

Runtimes y aceleración

Sistema → Runtimes nativos lista las variantes recomendadas (★) y alternativas. Por ejemplo, con iGPU Intel puedes instalar `openvino` o `vulkan` y compararlas: el autoajuste registra el rendimiento de cada sesión. Los instaladores traen llama-server y sd-cli precompilados en `bundled/runtimes/` para CPU, Vulkan y, según la plataforma, CUDA o Metal (marcados «✓ incluido»); Nexus-Office prueba primero la variante de GPU de tu hardware y cae a la CPU si falta el driver. Ver Plataformas. `python -m nexus setup --auto --all` instala también PyTorch acelerado (MusicGen) si hay GPU.

App Android autónoma: no hay que instalar nada. La APK trae llama-server de CPU y otro con GPU y NPU (Hexagon para la NPU de los Snapdragon 8 Gen 2 o posteriores, OpenCL para Adreno, Vulkan para el resto); Nexus-Office detecta CPU, GPU y NPU (`GET /api/hardware`), pone el modelo en la NPU o en la GPU si llama.cpp puede usarlas y el autoajuste (`GET /api/android/autoajuste`, campos `gpu` y `accel`) compara GPU y CPU con respuestas reales y se queda con lo más rápido. Si la NPU falla pasa a la GPU, y si falla la GPU sigue en CPU; lo anota en `data/android_accel.json` (`npu_broken`, `gpu_broken`; para volver a probarlas, borra ese fichero y `data/android_tune.json`). Ver la app autónoma.

Línea de comandos

```
python -m nexus run [--host 0.0.0.0] [--port 8775] [--no-browser]
python -m nexus detect                 # informe de hardware + perfil
python -m nexus setup --auto [--all]   # extras según hardware
python -m nexus pull auto|<id>         # modelos recomendados
python -m nexus modelos [--descargar] [--todos]   # evalúa el hardware, elige el modelo de cada tipo y lo descarga
si hay internet
python -m nexus servicio instalar|quitar|estado [--red] [--puerto P]   # arranque automático (ver SERVICIO.md)
python -m nexus oficina [<perfil>|--quitar]       # perfil de oficina (ver OFICINA.md)
python -m nexus chat [--agent id]      # chat en terminal
python -m nexus wheels --check         # comprueba que wheelhouse/ instala sin internet
python -m nexus cleanup [--dry-run] [--no-keep]   # quita del instalador lo que este equipo no usa (ver
PLATAFORMAS.md)
```

Modelo de IA elegido al instalar

`python -m nexus modelos` (lo llaman todos los instaladores) lee el perfil de hardware y, de `nexus/models/recommended.py`, toma el modelo marcado como mejor (`best`) de cada tipo: chat y visión (Qwen3-VL Thinking 2B, 4B u 8B), documentos (Qwen3-Embedding 0.6B), voz (Piper) e imagen (solo con `--todos`). Imprime el equipo (RAM, VRAM, acelerador, gama), lo elegido y los modelos que caben pero irían lentos. Con `--descargar` comprueba la conexión con Hugging Face y los baja; sin internet solo recomienda. Los instaladores lo controlan con `--modelos=auto|todos|no` / `-Modelos` / `/MODELOS=`. Los modelos descargados se activan solos si no había otro del mismo tipo.

Servicio (arranque automático)

`nexus/service.py` registra `python -m nexus run --no-browser --host H --port P` en el gestor de servicios del sistema: systemd (del sistema como root, con `User=` el dueño de la carpeta o `SUDO_USER`; de usuario con `loginctl enable-linger` si no), cron `@reboot` sin systemd, launchd (LaunchDaemon como root, LaunchAgent si no), el Programador de tareas de Windows (tarea «Nexus-Office»: al arrancar el equipo como SYSTEM si eres administrador, al iniciar sesión si no; `pythonw.exe`, reinicio ante fallos, sin límite de tiempo y regla de firewall con `--red`) o termux-services + Termux:Boot. Con `sudo`, si `NEXUS_DATA_DIR` apunta a `/root`, el servicio usa los datos del usuario real. Si Nexus-Office ya responde en el puerto, `nexus run` solo abre el navegador.

En la interfaz: *Sistema → Arranque automático* (`GET/POST /api/service`, cambiarlo exige modo administrador). El servicio se instala con los permisos del proceso de Nexus-Office: para que arranque con el equipo, instálalo desde el instalador, con `sudo` o desde una consola de administrador. Todo en SERVICIO.md.

Descargas para la oficina

Copia los instaladores (las carpetas `windows/`, `linux/`, `macos/`, `android/` de una entrega) en `data/descargas/` y aparecen en *Sistema → Descargas* (`GET /api/downloads`; ficheros en `/descargas/<ruta>`). `nexus/downloads.py` los clasifica por plataforma según el nombre, marca el del sistema anfitrión y la página marca el del dispositivo que la mira. Desde la red hace falta la clave de red, como en el resto de rutas.

Construir los instaladores

`python packaging/build.py [--targets …] [--apk …] [--docs …] [--keep]` genera `dist/{windows,linux,macos,android}`, el zip de código fuente y `SHA256SUMS.txt`. Antes hay que preparar `.cache/desktop-runtimes` con `build_python_runtimes.py`, `build_desktop_runtimes.sh` y `build_gpu_runtimes.py`, y tener `makensis` para el `.exe`. Paso a paso en Plataformas → Empaquetado.

Datos

Todo vive en `data/` (con el `.deb`, en `~/.local/share/nexus-office`; en general, donde indique `NEXUS_DATA_DIR`): `config.yaml`, `nexus.db` (memoria, grafo, salón, auditoría), `models/`, `runtimes/`, `outputs/`, `agents/*.yaml`, `tools/community|proposals/`, `logs/` (con `servicio.log` del servicio en Windows, macOS y cron), `descargas/` (instaladores para la red), `perf_history.json`, `tuning.json`. Haz copia de esa carpeta para respaldar a Nexus-Office.

Nexus-Office como servicio: arranque automático

Instalado como **servicio**, Nexus-Office arranca solo al encender el equipo, sin ventana, y se reinicia si falla. Es lo normal en el PC de la oficina: los móviles y los otros equipos lo encuentran siempre en marcha, aunque nadie haya abierto la aplicación.

- Funciona en Windows, Linux (también Raspberry Pi), macOS, Android con Termux y la app **Nexus-Office autónomo**.
- El servicio apunta a la carpeta donde está instalado Nexus-Office. La carpeta sigue siendo portable, pero si la mueves, vuelve a instalar el servicio.
- Con el servicio en marcha, el acceso directo, `start.bat` o `./start.sh` **no arrancan un segundo Nexus-Office**: ven que ya responde y solo abren el navegador en `http://127.0.0.1:8775`.

Cómo se activa

Desde	Cómo
Instalador de Windows (`.exe`)	Página *Opciones* → **Arrancar Nexus-Office como servicio al encender el equipo** y, si quieres, **Accesible desde la red local**. Solo ese paso pide permiso de administrador (UAC). Silencioso: `/S /SERVICIO /RED`.
Instalador de Linux (`.run`) o `install.sh`	`--servicio`, y `--red` para la red local. Crea un servicio de usuario. Ejecuta el `.run` sin `sudo` (con `sudo` se instalaría en la carpeta de root) y, si quieres el servicio del sistema, después `sudo <carpeta>/venv/bin/python3 -m nexus servicio instalar`.
Paquete `.deb`	`sudo nexus-office servicio instalar [--red]`
macOS	`./install.sh --servicio [--red]`, o el comando de la terminal (abajo).
`install.ps1` / `install.bat`	`-Servicio`, y `-Red` para la red local.
Termux	`bash install-termux.sh --servicio` (siempre accesible desde la red local).
Interfaz web	*Sistema → Arranque automático* → **INSTALAR SERVICIO** (modo administrador). Marca *accesible desde la red local* si lo vas a usar desde móviles u otros PC.
App Nexus-Office autónomo	*Sistema → Arranque automático* → **ARRANCAR AL ENCENDER**.

También con la variable de entorno `NEXUS_SERVICIO=1` en `install.sh` e `install.ps1`.

Comandos

En Windows, `venv\python.exe`; en Linux y macOS, `venv/bin/python3`; con el `.deb`, `nexus-office` en lugar de `python -m nexus`.

```
python -m nexus servicio instalar             # arranque automático, escuchando solo en este equipo
python -m nexus servicio instalar --red       # escucha en la red local (0.0.0.0): móviles y otros PC
python -m nexus servicio instalar --puerto 9000
python -m nexus servicio estado               # tipo, dónde está registrado y si responde
python -m nexus servicio quitar               # lo quita (Nexus-Office sigue instalado)
```

Sin `--red` se usa la dirección de `server.host` de `data/config.yaml` (por defecto `127.0.0.1`), y sin `--puerto`, `server.port` (8775). `--host <ip>` fija otra dirección de escucha.

Desde la red, los dispositivos que no son el propio equipo necesitan la **clave de red** (*Red COOP → Conectar dispositivos*). Ver Acceso desde otros dispositivos.

Qué hace en cada sistema

Sistema	Con permisos de administrador	Sin permisos
Windows	Tarea programada «Nexus-Office» que arranca **con el equipo** (30 s después), con la cuenta SYSTEM. Con `--red`, abre el puerto en el Firewall de Windows (regla «Nexus-Office»).	Tarea programada que arranca **al iniciar sesión** tu usuario. Sin regla de firewall: con `--red`, Windows puede preguntar la primera vez si permite el acceso.

Sistema	Con permisos de administrador	Sin permisos
Linux con systemd	Servicio del sistema `/etc/systemd/system/nexus-office.service`, que corre como el usuario dueño de la carpeta (o el que hizo `sudo`). Arranca con el equipo.	Servicio de usuario `~/.config/systemd/user/nexus-office.service` y `loginctl enable-linger`, para que arranque al encender sin iniciar sesión. Si *linger* no se puede activar, arranca al iniciar sesión.
Linux sin systemd	Línea `@reboot` en el crontab.	Igual.
macOS	LaunchDaemon `/Library/LaunchDaemons/ai.nexus.office.server.plist`: arranca con el equipo.	LaunchAgent `~/Library/LaunchAgents/ai.nexus.office.server.plist`: arranca al iniciar sesión.
Android (Termux)	—	`termux-services` (`$PREFIX/var/service/nexus-office`) y, con la app **Termux:Boot**, arranque al encender el móvil (`~/.termux/boot/nexus-office.sh`, con *wake lock*).
App Nexus-Office autónomo	—	El servicio en primer plano de la app se lanza al encender el móvil y tras actualizar la app.

En todos los casos:

- Se reinicia solo si se cae: systemd a los 5 s, launchd a los 10 s, el Programador de tareas cada minuto (hasta 999 veces).
- No tiene límite de tiempo: en Windows la tarea no se para a las 72 h ni al pasar a batería.
- Arranca con `python -m nexus run --no-browser`, sin abrir el navegador.
- En Windows usa `pythonw.exe`, así que no abre ninguna ventana.

Para que el arranque sea **con el equipo** y no al iniciar sesión, instálalo con permisos de administrador: desde el instalador de Windows (pide UAC), con `sudo` en Linux y macOS, o desde una consola de administrador en Windows. Si lo instalas desde la interfaz web, se instala con los permisos de Nexus-Office (normalmente, los de tu usuario).

Con el `.deb`, `sudo nexus-office servicio instalar` crea el servicio del sistema, que corre como tu usuario y usa tus datos de `~/.local/share/nexus-office`.

App Nexus-Office autónomo (Android)

En *Sistema → Arranque automático*, pulsa **ARRANCAR AL ENCENDER**. Desde entonces, al encender el móvil (después del primer desbloqueo) se inicia el servicio en primer plano de Nexus-Office, con su notificación, sin abrir la app. **DESACTIVAR** lo deja como antes: Nexus-Office arranca al abrir la app.

Algunos fabricantes bloquean el arranque de las apps con el ahorro de batería extremo. Si no arranca solo, quita a Nexus-Office del ahorro de batería en los ajustes de Android. Si el sistema lo impide, Nexus-Office arranca igualmente al abrir la app.

Quitar el servicio

- **Interfaz web**: *Sistema → Arranque automático* → **QUITAR** (modo administrador).
- **Terminal**: `python -m nexus servicio quitar`. Si lo instalaste con permisos de administrador, quítalo también con ellos: `sudo` en Linux y macOS (`sudo nexus-office servicio quitar` con el `.deb`), o una consola de administrador en Windows.
- **Desinstalar**: el desinstalador de Windows y la desinstalación del `.deb` (`sudo apt remove nexus-office`) quitan el servicio antes de borrar Nexus-Office. En Windows, si la tarea era del sistema, pide permiso de administrador.
- **App Nexus-Office autónomo**: **DESACTIVAR** en *Sistema → Arranque automático*.

Quitar el servicio no borra nada: Nexus-Office sigue instalado con sus datos y se abre como siempre.

Registro (logs)

Sistema	Dónde
Windows	`data\logs\servicio.log` (con `pythonw.exe` no hay consola: la salida va a ese fichero, también al abrir Nexus-Office desde el acceso directo).
macOS	`data/logs/servicio.log`

Sistema	Dónde
Linux con systemd	`journalctl -u nexus-office` (del sistema) o `journalctl --user -u nexus-office` (de usuario).
Linux con cron	`data/logs/servicio.log`
Termux	`data/logs/current` (svlogd)

Con el `.deb`, `data/` es `~/.local/share/nexus-office`. El registro general de Nexus-Office (`data/logs/`) se ve también en *Sistema → Registro* (modo administrador).

Problemas frecuentes

- **`servicio estado` dice «en marcha: no»**: mira el registro. Si Nexus-Office se acaba de instalar, el primer arranque tarda más; en Windows, además, la tarea del sistema espera 30 s tras encender.
- **Solo arranca al iniciar sesión**: se instaló sin permisos de administrador. Quítalo y vuelve a instalarlo con `sudo` (Linux, macOS) o desde una consola de administrador (Windows). En Linux, el aviso de `loginctl enable-linger` indica lo mismo.
- **«el servicio del sistema se quita con sudo»** o **«no se pudo borrar la tarea»**: el servicio se instaló con permisos de administrador; quítalo con ellos.
- **Los móviles no conectan**: instálalo con `--red` (o marca *accesible desde la red local*), comprueba que usan la clave de red y, en Windows sin permisos de administrador, permite Nexus-Office en el firewall.
- **El puerto 8775 está ocupado**: otro programa (o un Nexus-Office abierto a mano antes de instalar el servicio) lo usa. Cierra Nexus-Office desde *Sistema → Apagar Nexus-Office* o instala el servicio con `--puerto`.
- **Has movido la carpeta**: el servicio sigue apuntando a la ruta antigua. `servicio quitar` y `servicio instalar` desde la carpeta nueva.
- **Linux sin systemd ni cron** (algunos contenedores): no hay servicio posible; arranca Nexus-Office con `./start.sh --no-browser`.
- **Termux dice que falta termux-services**: `pkg install termux-services`, reinicia Termux y `sv-enable nexus-office`. Para arrancar al encender el móvil, instala también la app Termux:Boot desde F-Droid y ábrela una vez.
- **Parar Nexus-Office sin quitar el servicio**: *Sistema → Apagar Nexus-Office* lo para hasta el siguiente arranque. Con el servicio de Windows o de Linux, si se cierra con error se vuelve a lanzar solo.

API

Base: `http://127.0.0.1:8775`. Los endpoints de administración requieren la cabecera `X-Admin-Token` (obtenida con `/api/admin/setup` o `/api/admin/login`). Documentación interactiva generada por FastAPI en `/docs`.

WebSocket `/ws`

Enviar: `{"type": "chat", "text": "...", "agent": "auto", "session_id": "...", "images": ["..."]}`

Eventos recibidos: `session`, `agent`, `state` (`thinking|tool|idle`), `token`, `tool_call`, `tool_result`, `approval_required`, `approval_resolved`, `learned`, `media`, `graph_update`, `done` (`message_id`, `text`), `error`, `lounge`, `coop`.

REST principal

Método y ruta	Descripción
`GET /api/status`	Estado general (perfil, LLM, voz, memoria, red, pendientes).
`GET/PUT /api/config`	Configuración (claves `user.*` en modo usuario; todas en admin).
`POST /api/admin/setup` · `login` · `logout` · `pin` · `shutdown`, `GET /api/admin/logs`	Administración.
`GET /api/hardware`, `GET /api/monitor`, `POST /api/hardware/redetect`	Hardware, monitor en vivo, re-detección.
`GET /api/runtimes`, `POST /api/runtimes/install`	Runtimes nativos.
`GET /api/sessions`, `GET/DELETE /api/sessions/{id}`	Conversaciones.
`POST /api/feedback`	`{message_id, score: 1|-1, comment}`.
`GET/POST /api/memory`, `GET /api/memory/search?q=`, `DELETE /api/memory/{id}`	Memoria.
`GET /api/graph?scope=` (`*` = general con el subgrafo de cada agente, por defecto; vacío = solo el general; id de agente = solo su grafo individual), `GET /api/graph/scopes` (tamaño y documentos de cada grafo), `DELETE /api/graph/{id}`	Grafo de conocimiento.
`POST /api/image/generate`, `POST /api/image/analyze` (multipart), `POST /api/music/generate`, `GET /api/outputs`	Multimedia.
`GET /api/voice`, `POST /api/voice/tts` (WAV o 204 = usar navegador), `POST /api/voice/stt` (multipart)	Voz.
`GET /api/rag`, `GET /api/rag/search?q=`, `GET /api/rag/formats` (formatos admitidos y herramientas de descompresión disponibles), `POST /api/rag/upload` (multipart; documentos y archivos ZIP/RAR/7z/TAR), `POST /api/rag/import` (multipart, varios `files`; devuelve `{results, ok, failed, seconds}`), `POST /api/rag/url` `{url}`, `POST /api/rag/crawl` `{url, max_pages, max_depth, scope}` (rastreo de documentación), `GET /api/rag/{id}/file`, `DELETE /api/rag/{id}`. Destino: campo `agent` en upload, import, url y crawl (vacío = grafo general; id = solo ese agente); `PATCH /api/rag/{id}` `{agent}` lo mueve; `GET /api/rag?agent=` y `/api/rag/search?agent=` filtran como lo vería ese agente	Biblioteca RAG y URL → PDF.
`POST /api/avatar/upload` (GLB/GLTF/VRM)	Modelo 3D propio para el avatar.
`GET/PUT /api/agents`, `DELETE /api/agents/{id}`	Agentes.
`GET /api/lounge`, `POST /api/lounge/meet`, `POST /api/lounge/say`, `POST /api/lounge/proposals/{id}`	Salón.
`GET /api/tools`, `POST /api/approvals/{id}`, `GET /api/tools/audit`, `GET/POST/PUT /api/proposals…`	Herramientas.
`GET /api/models`, `/recommended`, `/search?source=&q=`, `/files?source=&repo=`; `POST /api/models/download`, `/recommended/{id}`, `/import`, `/activate/{id}`, `/activate-spec`	Modelos.
`GET /api/coop`, `POST /api/coop/create|join|start|stop|chat|ping/{n}|agent`, `GET /api/coop/invite`	Red cooperativa.

Método y ruta	Descripción
`GET /api/vpn`, `POST /api/vpn/enable {enabled, port}`, `POST /api/vpn/peers {name, lookup_public_ip}`, `DELETE /api/vpn/peers/{id}`, `GET /api/vpn/server.conf`, `POST /api/vpn/apply`, `POST /api/vpn/upnp` *(admin)*	VPN WireGuard para la app Nexus-Office VPN (ver VPN.md).
`GET /api/office/profiles`, `POST /api/office/profile {id, reset_agents}` *(admin; `id` vacío quita el perfil)*	Perfiles de oficina para pymes (ver OFICINA.md).

Servicio y descargas

Método y ruta	Descripción
`GET /api/service`	Arranque automático: `{kind, registered, supported, admin_os, port}`. `kind`: `systemd`, `cron`, `launchd`, `windows` o `termux`; `registered`: dónde está registrado (vacío si no lo está); `admin_os`: si Nexus-Office corre con permisos de administrador del sistema (con ellos arranca con el equipo, sin ellos al iniciar sesión). En la app Android autónoma devuelve `{kind: "android", supported: false}`: allí lo gestiona la propia app.
`POST /api/service {action: "instalar"|"quitar", red}` *(admin)*	Instala o quita el servicio con el puerto de `server.port`. `red: true` escucha en `0.0.0.0`; si no, usa `server.host`. Devuelve `{ok, detail}`; `400` con el motivo si falla. Ver SERVICIO.md.
`GET /api/downloads`	Instaladores de `<datos>/descargas/`: `{folder, host, host_label, files: [{name, path, size, platform, label}]}`. `platform`: `windows-x64`, `windows-arm64`, `linux-x86_64`, `linux-aarch64`, `macos-arm64`, `macos-x86_64`, `android`, `codigo`, `docs` u `otros`; `host` es la del equipo donde corre Nexus-Office.
`GET /descargas/{path}`	Descarga un fichero de `<datos>/descargas/` (`path` como lo da `/api/downloads`; las APK con su tipo MIME de Android). `404` si no existe o sale de la carpeta.
`GET /api/package/termux`	`Nexus-Office-<versión>-termux.tar.gz` con el código para instalar en Termux (lo usa la app Nexus-Office VPN).

Desde otro dispositivo de la red, estas rutas exigen la clave de red, como todas salvo `/api/ping`.

Gestor de conocimiento

`scope`: vacío o `general` = grafo general; id de agente = su subgrafo; `*` o sin indicar = todos. Las escrituras exigen modo administrador. Ver MEMORIA.md.

Método y ruta	Descripción
`GET /api/kg/overview`	Categorías, tamaño de cada grafo por categoría, recuerdos por ámbito, especialidades, embeddings y trabajos.
`GET /api/kg/nodes?scope=&q=&category=&type=&limit=&offset=&semantic=`	Buscar nodos. `semantic=true` busca por significado con el modelo de embeddings.
`GET /api/kg/nodes/{id}`	Un nodo con sus relaciones.
`POST /api/kg/nodes {name, type, category, scope, note}`	Crear un nodo.
`PATCH /api/kg/nodes/{id} {name, type, category, weight, note, scope}`	Editar. Con `scope`, lo mueve a otro grafo; si ya existe allí, se fusionan.
`DELETE /api/kg/nodes/{id}`	Borrar un nodo y sus relaciones.
`POST /api/kg/merge {src, dst}`	Fusionar `src` en `dst` (mismo grafo).
`POST /api/kg/relations {src, dst, rel, weight}` (ids de nodo), `DELETE /api/kg/relations/{id}`	Relaciones. No se permiten entre dos agentes distintos.
`POST /api/kg/import` (multipart `file` o varios `files`, `target`: vacío, id de agente o `auto`)	Insertar un grafo de cualquier formato (se detecta por el contenido); varios ficheros se unen en uno, p. ej. `nodes.csv` + `edges.csv`. Trabajo en segundo plano; el informe trae `format`, `nodes`, `new`, `merged`, `relations` y `by_scope`. Ver GRAFOS.md.
`GET /api/kg/formats`	Formatos de grafo admitidos y sus extensiones.
`POST /api/kg/archive` (multipart `file`, `target`=`auto`, `reason`, `rag`, `max_reason`)	Leer, razonar y procesar un ZIP, RAR, 7z o TAR. Trabajo en segundo plano.
`GET /api/kg/export?scope=&format=json|graphml|csv`	Exportar un grafo.
`POST /api/kg/reorganize {apply}`	Propuesta de reorganización; con `apply: true`, la aplica.
`POST /api/kg/duplicates {scope}`	Buscar nodos duplicados.

Método y ruta	Descripción
`GET /api/kg/jobs`, `GET /api/kg/jobs/{id}`	Trabajos y su informe. El WebSocket emite `{"type": "kg_job", "job": …}` con cada avance.
`GET /api/kg/memories?scope=&kind=`, `PATCH /api/kg/memories/{id} {scope}`	Recuerdos por ámbito y moverlos.
`GET /api/kg/embeddings`, `POST /api/kg/embeddings/model {model}`, `POST /api/kg/embeddings/reindex`	Modelo de embeddings, progreso del indexado, activar y reindexar.

Acceso remoto

Puerto remoto: `https://IP:8443` (TLS 1.3). Ahí todas las rutas, salvo `/`, `/api/ping`, `/api/remote/pair` y `/static/…`, exigen la llave del dispositivo, en la cookie `nexus_dev` o en la cabecera `X-Nexus-Device`. El WebSocket `/ws` también la exige. Sin llave se recibe `401`.

Método y ruta	Descripción
`POST /api/remote/pair {code, name}`	Canjea un código de un solo uso. Devuelve `{device, token}` y fija la cookie segura. Tras 8 fallos en 15 minutos responde `429`.
`GET /api/remote` *(admin)*	Estado: activado, escuchando, huella, direcciones, UPnP, CGNAT, dispositivos.
`POST /api/remote/enable {enabled, port}` *(admin)*	Activa o desactiva el puerto remoto (requiere reiniciar).
`POST /api/remote/pairing {name}` *(admin)*	Código de un solo uso (10 minutos), `invite NEXUSR1:…` para la app y enlaces `?pair=`.
`DELETE /api/remote/devices/{id}` *(admin)*	Revoca un dispositivo y corta sus conexiones.
`POST /api/remote/upnp {open}` *(admin)*	Abre o cierra el puerto en el router por UPnP y lee la IP pública.
`POST /api/remote/public-ip` *(admin)*	IP pública vista desde internet (ipify) y detección de CGNAT.
`POST /api/remote/certificate` *(admin)*	Certificado nuevo (cambia la huella anclada).
`GET /api/remote/router` *(admin)*	Asistente del router: puerta de enlace, IP y MAC de este equipo, URLs de configuración, marca y ruta del menú, UPnP-IGD y la regla exacta.
`POST /api/remote/check` *(admin)*	IP pública, CGNAT y si el puerto responde en la IP pública.

Herramientas: sistema modular y supervisado

Anatomía de una herramienta

```
TOOL = {
    "name": "convertir_unidades",
    "description": "Convierte entre unidades de longitud",
    "parameters": {"type": "object",
                   "properties": {"valor": {"type": "number"}, "de": {"type": "string"}, "a": {"type": "string"}},
                   "required": ["valor", "de", "a"]},
    "risk": "low",                 # low | medium | high
    "permissions": [],             # network, filesystem, code_exec, system, memory, media, agents
    "timeout": 30,                 # opcional, segundos
    "version": "1.0",
}

def run(args, ctx):
    factores = {"m": 1, "km": 1000, "cm": 0.01, "mi": 1609.344}
    return {"resultado": args["valor"] * factores[args["de"]] / factores[args["a"]]}
```

- **Integradas**: `nexus/tools/builtin/*.py` (pueden declarar varias con `TOOLS = [(manifiesto, función), ...]`).
- **Comunidad**: `data/tools/community/*.py`, solo tras aprobación. Su manifiesto se lee por AST (sin ejecutar el código) y se ejecutan en un **subproceso aislado** (`python -I`, entorno mínimo, directorio sandbox, límite de tiempo).

Supervisión

Política	Comportamiento
`auto`	Se ejecuta directamente.
`ask`	Tarjeta de aprobación en la UI para el usuario.
`admin`	Solo un administrador puede aprobarla.
`deny`	Bloqueada.

Por defecto: riesgo bajo `auto`, medio `ask`, alto `admin`. Las peticiones que llegan de otros nodos de la red se **elevan un nivel**. Todo queda en la auditoría (agente, argumentos, decisión, aprobador, estado, duración).

Herramientas incluidas

`calculator`, `datetime_now`, `memory_search`, `memory_save`, `knowledge_graph` (hechos y atributos de una entidad), `graph_import` (importa como conocimiento un grafo de cualquier formato desde el sandbox o desde un texto, al grafo general, al del agente o repartido; ver GRAFOS), `system_info`, `web_search`, `web_fetch` (bloquea direcciones de red local; pagina el texto largo con `offset`/`next_offset`), `url_to_pdf` y `crawl_docs` (rastreo de un sitio de documentación hacia la biblioteca RAG; los dominios grandes van por lotes reanudables, ver RAG), `rag_search`, `rag_list`, `file_list`/`read`/`write` (confinadas a `data/sandbox`; `file_read` lee por tramos con `offset`/`max_chars` y `file_list` pagina de 500 en 500; `file_list` acepta una subcarpeta, p. ej. `proyectos/<nombre>` de un ZIP importado), `python_exec` (alto riesgo), `generate_image`, `compose_music`, `delegate`, `list_agents`, `propose_tool`, y las de la mesa de bolsa simulada (`bolsa_mercados`, `bolsa_cotizacion`, `bolsa_cartera`, `bolsa_costes`, `bolsa_orden`, `bolsa_cancelar`, `bolsa_informe_fiscal`; ver BOLSA), y las de edición de imágenes con GIMP (`imagen_editar`, `imagen_info`, `gimp_estado` y `gimp_script`, esta última de alto riesgo; ver GIMP).

Propuestas de los agentes

1. Un agente (normalmente el **Forjador**) detecta una carencia y llama a `propose_tool` con el código.
2. **Análisis estático**: sintaxis, manifiesto, `run(args, ctx)`, importaciones y llamadas peligrosas (`subprocess`, `os`, `socket`, `eval`, `exec`, `open`, `rmtree`…) → riesgo sugerido.
3. El administrador revisa en **Herramientas → Propuestas**: puede editar el código (se re-analiza), **probarlo en el sandbox** con argumentos de prueba, aprobarlo, rechazarlo o compartirlo con la red.
4. Al aprobar, si el análisis sugiere un riesgo mayor que el declarado, **se eleva automáticamente**.

5. El motivo de un rechazo se guarda como *lección* que el agente verá en el futuro.

Memoria y gestor de conocimiento

Desde la 1.6.0, Nexus-Office separa lo que es **común** a todos los agentes de lo que es **saber hacer propio** de cada uno.

Cómo se organiza

Dónde	Qué guarda	Categorías
Grafo general (común)	Cómo eres y qué te gusta, las pautas de comportamiento que pides, tus relaciones con otras personas, cómo interactúas con los agentes y el conocimiento general que sirve a cualquiera para razonar.	`perfil`, `preferencia`, `comportamiento`, `relacion`, `interaccion`, `comun`
Subgrafo aislado de cada agente	El saber hacer de su especialidad: conceptos técnicos, procedimientos paso a paso, sus documentos y su experiencia. Ningún otro agente lo ve.	`especialidad`, `procedimiento`, `documento`, `experiencia`

- Los gustos son comunes: si le dices al artista que prefieres respuestas cortas o que no uses emojis, el programador también lo sabrá.
- Las **pautas de comportamiento** («no uses emojis», «respóndeme siempre con ejemplos», «llámame Joseba») se guardan como nodos `comportamiento` y recuerdos de tipo `behavior`. Todos los agentes las reciben en su prompt, en la sección *Pautas de comportamiento que el usuario te ha pedido*.
- Las **relaciones** («mi hermana Ane es arquitecta») y las **interacciones** (con qué agente hablas, en quién delega cada agente) van al general.
- Cada subgrafo de agente puede enlazar con nodos del general (puentes). Nunca enlaza con el subgrafo de otro agente.

El organizador

`nexus/memory/organizer.py` decide dónde va cada cosa que se aprende, en la conversación, con `memory_save`, al importar grafos o al leer un ZIP:

1. Busca, con reglas, datos personales, pautas, relaciones e interacciones. Todo eso va al general.
2. Compara el resto con el **perfil de especialidad** de cada agente: su descripción, el campo `specialty` (*Agentes → Editar → Especialidad*) y sus palabras clave. - Con un modelo de embeddings, usa la afinidad semántica. El texto va al especialista si su afinidad supera `memory.organizer.min_affinity` (0.35) y gana al conocimiento general por `margin` (0.03). - Sin modelo, cuenta las palabras clave de la especialidad que aparecen.
3. Si nada encaja, va al general como `comun`. Si el texto describe pasos («primero…, luego…, finalmente…»), se guarda como `procedimiento`.

El agente generalista `nexus` no tiene especialidad: lo que aprende en sus conversaciones se reparte entre los especialistas o el general. Si no hay ningún especialista definido, cada agente guarda lo suyo como antes.

Gestor de conocimiento (Ajustes, modo administrador)

Pestaña	Para qué
Resumen	Nodos, relaciones, documentos y recuerdos de cada grafo, por categoría, con la especialidad de cada agente.
Explorar y editar	Filtrar por grafo, categoría y tipo, y buscar por texto o **por significado**. Editar nombre, tipo, categoría, peso y nota. **Mover** un nodo a otro grafo (sus relaciones se recalculan y las que cruzarían agentes se quitan), **fusionar** dos nodos, añadir o borrar relaciones y crear nodos.
Importar / exportar	Insertar otros grafos de conocimiento en el general, en un agente o en **automático** (cada nodo a su sitio). Exportar cualquier grafo en JSON, GraphML o CSV.
Leer un ZIP	Ver más abajo.
Mantenimiento	**Reorganizar** lo aprendido: revisa cada nodo y recuerdo, propone qué mover y lo aplica cuando confirmas. **Duplicados**: nodos del mismo grafo que significan lo mismo, con botón para fusionarlos. **Recuerdos por ámbito**, para moverlos de uno a otro.
Embeddings	Modelo activo, progreso del indexado, **reindexar** y descarga del modelo recomendado.

Los trabajos largos (importar, leer un ZIP, reorganizar, buscar duplicados) van en segundo plano y muestran su progreso abajo, sin bloquear el chat.

Formatos de grafo que se pueden importar

Cualquier tipo y formato, reconocido por el contenido: JSON en todas sus variantes (node-link de D3, vis.js o NetworkX, Cytoscape, JSON Graph Format, JSON-LD, GraphSON, Neo4j, Arrows.app, Obsidian Canvas, tripletas), CSV/TSV y hojas Excel/ODS (aristas, nodos, matriz de adyacencia), GraphML, GEXF, GML, DOT, Pajek, GDF, UCINET, TGF, RDF (Turtle, N-Triples, N-Quads, TriG, RDF/XML, OWL), Cypher de Neo4j, Mermaid, PlantUML, mapas mentales (FreeMind, XMind, OPML), notas con `[[enlaces]]`, YAML, TOML, bases SQLite y cualquiera de ellos comprimido o varios en un ZIP. Lista completa, cómo se importa desde la app, la terminal o el chat, y qué se guarda de cada grafo: GRAFOS.md.

Lo importado se fusiona con lo que ya hay: los nodos con el mismo nombre se unen y las relaciones repetidas se refuerzan. Los atributos de cada nodo van a su nota y los agentes los usan al responder. Si el fichero trae una categoría o un ámbito por nodo, se respetan. En modo automático, los nodos sin ámbito los reparte el organizador.

Leer un ZIP (también RAR, 7z y TAR)

1. Lo descomprime de forma segura (sin rutas fuera de la carpeta, con límites de tamaño y número de ficheros).
2. Inserta los grafos que traiga (JSON, GraphML, CSV, RDF…) y convierte en grafo las notas con `[[enlaces]]`.
3. Lee hasta 600 documentos, con los README e índices primero, y elige el destino. En automático, el saber hacer va al especialista y lo común al general.
4. Lo añade a la biblioteca RAG, si está marcado.
5. **Agrupa los documentos por temas** con el modelo de embeddings, hasta 12 temas.
6. **Razona** con el LLM los documentos más representativos de cada tema: `memory.zip_reason_docs` (12 por defecto) o lo que indiques en la pestaña. De cada uno saca un resumen, hechos clasificados y procedimientos paso a paso. Los hechos van a su categoría, y los procedimientos quedan como nodos `procedimiento` y recuerdos `skill` del agente.
7. Escribe una síntesis del archivo completo. Sin LLM, usa un resumen extractivo y las reglas.

El informe muestra los temas, qué fue a cada grafo y lo que se aprendió.

Modelo de embeddings: Qwen3-Embedding 0.6B

Los embeddings convierten un texto en un vector que representa su significado. Nexus-Office los usa para buscar en la biblioteca y en la memoria, decidir qué es común y qué es de cada agente, agrupar documentos por temas y encontrar duplicados.

- **Modelo**: `qwen3-embedding-0.6b`, de `Qwen/Qwen3-Embedding-0.6B-GGUF`, fichero `Qwen3-Embedding-0.6B-Q8_0.gguf` (639 MB). Es multilingüe (más de 100 idiomas, español incluido), admite textos de hasta 32k tokens y está entrenado para buscar y clasificar documentación. Recibe **instrucciones por tarea**: buscar, recordar, clasificar, organizar, deduplicar.
- **Descarga**: en *Modelos* (recomendado ★ para todos los perfiles), en *Ajustes → Gestor de conocimiento → Embeddings* o con `python -m nexus pull qwen3-embedding-0.6b`. Se descarga una vez y queda en `data/models/`, disponible sin conexión. Si es el único de su tipo, se activa solo.
- **En CPU**: corre en un `llama-server` aparte (`--embedding --pooling last`), con el mismo runtime llama.cpp que el chat, incluido en todos los instaladores. Si no hay runtime, usa `llama-cpp-python`. Se arranca al primer uso y se para tras 10 minutos sin peticiones, para liberar RAM.
- **Indexado progresivo**: en un PC modesto (por ejemplo, un i3 sin GPU), calcular los vectores de una biblioteca grande lleva un rato. Por eso los documentos y recuerdos nuevos se indexan al momento con vectores rápidos (hashing). Un proceso de fondo los recalcula con el modelo en lotes de 4 y cede el turno mientras el LLM genera una respuesta o hay una búsqueda esperando. La búsqueda mezcla los ya recalculados y los pendientes, así que todo se encuentra desde el primer momento.
- **Cambiar de modelo** reindexa en segundo plano, con el mismo mecanismo.
- **Ahorrar RAM**: `memory.embed_dims: 512` (o 256) recorta los vectores (Matryoshka), con poca pérdida de precisión.

Configuración (`data/config.yaml`)

```
memory:
  embedder: auto          # auto | gguf | hashing | ollama | sentence-transformers
  embed_model: qwen3-embedding-0.6b   # se rellena al activar el modelo
  embed_threads: 0        # hilos del servidor de embeddings (0 = los del LLM)
  embed_dims: 0           # recorte Matryoshka; 0 = completo (1024)
  organizer: {min_affinity: 0.35, margin: 0.03, min_keyword_hits: 1}
  zip_reason_docs: 12     # documentos de un ZIP que razona el LLM
```

API

Ver la sección *Gestor de conocimiento* en API.md.

Importar grafos como conocimiento

Nexus-Office puede incorporar a su conocimiento un grafo de **cualquier tipo y formato**: una red exportada de Gephi o Cytoscape, una base de Neo4j, una ontología RDF u OWL, un diagrama Mermaid, una hoja de Excel con nodos y relaciones, un mapa mental o la base de otro Nexus-Office. Todo funciona sin internet.

El formato se reconoce **por el contenido**. La extensión solo ayuda cuando el contenido no basta: un GraphML guardado como `.txt` o un JSON sin extensión se leen igual.

Lo importado se **fusiona** con lo que Nexus-Office ya sabe. Los nodos con el mismo nombre se unen en uno, las relaciones que ya existían se refuerzan y no se duplican, y cada nodo conserva su nota, su tipo y su categoría si ya los tenía. Desde ese momento los agentes lo usan al responder.

Cómo se importa

Desde la app

1. Entra en modo administrador y abre **Ajustes > Gestor de conocimiento > Importar / exportar**.
2. En **Insertar en**, elige dónde va lo importado: - **Automático**: cada nodo va a su sitio. Lo común (personas, lugares, preferencias) va al grafo general y el saber hacer de un oficio va al agente especialista. - **General**: para todos los agentes. - **Un agente**: solo para ese agente.
3. Suelta el fichero en el recuadro, o haz clic para elegirlo. Puedes soltar **varios ficheros a la vez**, por ejemplo `nodes.csv` y `edges.csv`, y se unen en un solo grafo.
4. La importación va en segundo plano y se ve su avance abajo. Al terminar, un aviso indica el formato reconocido, cuántos nodos son nuevos, cuántos se han fusionado con los que ya había y cuántas relaciones se han creado.

En **Formatos admitidos** se despliega la lista completa.

Desde la terminal

```
python -m nexus grafo formatos                                  # lista de formatos
python -m nexus grafo importar red.graphml                      # al grafo general
python -m nexus grafo importar nodes.csv edges.csv --en auto    # varios ficheros, cada nodo a su sitio
python -m nexus grafo importar recetas.ttl --en cocinero        # solo para el agente «cocinero»
python -m nexus grafo exportar copia.json --en todo             # exportar (json, graphml o csv con --formato)
```

Desde el chat

Los agentes tienen la herramienta `graph_import`. Puedes pedirles, por ejemplo, «importa como conocimiento el grafo `proveedores.gexf` de la carpeta de trabajo» o «guarda este diagrama como conocimiento: Pedido -> Albarán -> Factura». El agente lee un fichero de su carpeta de trabajo (`data/sandbox`) o el texto que le das, y lo guarda en el grafo general, en el suyo o repartido automáticamente.

Dentro de un ZIP

Al **leer un ZIP** (también RAR, 7z o TAR) en **Ajustes > Gestor de conocimiento > Leer un ZIP**, los grafos que trae se importan solos, en cualquiera de estos formatos, y el resto de documentos va a la biblioteca.

Formatos admitidos

Familia	Extensiones habituales	Qué se lee
Nexus-Office	`.json`, `.db`	Exportación propia y la base SQLite de otro Nexus-Office, respetando el grafo general y el de cada agente.
JSON node-link	`.json`	D3, vis.js, sigma.js, NetworkX (`node_link_data` y `adjacency_data`), JSON Graph Format, Arrows.app.
Cytoscape	`.cyjs`, `.cx`, `.cx2`, `.sif`, `.xgmml`	Cytoscape.js, CX y CX2, SIF y XGMML.
Neo4j	`.json`, `.jsonl`, `.csv`, `.cypher`	Exportación del Browser, API HTTP, APOC (JSON, Cypher y GraphML) y CSV de `neo4j-admin import` (`:ID`, `:START_ID`, `:END_ID`, `:TYPE`, `:LABEL`).

Familia	Extensiones habituales	Qué se lee
Cypher	`.cypher`, `.cql`	`CREATE`, `MERGE` y `MATCH … CREATE`, a mano o exportado. Las consultas (`MATCH … RETURN`) no crean nada.
JSON-LD	`.jsonld`, `.json`	schema.org, Wikidata y cualquier `@graph`.
GraphSON	`.json`, `.graphson`	TinkerPop, JanusGraph y Amazon Neptune, con tipos o sin ellos.
RDF	`.ttl`, `.nt`, `.nq`, `.trig`, `.n3`, `.rdf`, `.owl`	Turtle, N-Triples, N-Quads, TriG, N3, RDF/XML y ontologías OWL.
GraphML	`.graphml`	yEd (con sus etiquetas), Gephi, NetworkX, igraph y APOC.
GEXF	`.gexf`	Gephi, con sus atributos.
GML	`.gml`	igraph, NetworkX y Cytoscape.
Graphviz DOT	`.dot`, `.gv`	`digraph` y `graph`, con etiquetas y pesos.
Análisis de redes	`.net`, `.paj`, `.gdf`, `.dl`, `.tgf`, `.lgl`, `.ncol`	Pajek, GDF (GUESS/Gephi), UCINET DL, TGF, LGL y NCOL.
Tablas	`.csv`, `.tsv`, `.xlsx`, `.ods`, `.db`, `.sqlite`	Tablas de aristas (`source,target,relation`), de nodos (`id,label,…`), matrices de adyacencia, tripletas y ConceptNet. En Excel y ODS, cada hoja es una tabla. En SQLite, cada tabla.
Listas	`.edgelist`, `.adjlist`, `.txt`	`a b [peso]`, `A: B, C` y una tripleta por línea.
Diagramas	`.mmd`, `.puml`, `.txt`	Mermaid (flowchart, graph, erDiagram, classDiagram, mindmap), PlantUML y texto con flechas `A -> B : relación`.
Mapas mentales y esquemas	`.mm`, `.opml`, `.xmind`, `.md`	FreeMind y Freeplane, OPML, XMind y listas o títulos con sangría.
Notas enlazadas	`.md`, `.canvas`, `.zip`	Obsidian, Logseq y Foam (`[[enlaces]]`) y Obsidian Canvas.
YAML y TOML	`.yaml`, `.yml`, `.toml`	Con nodos y aristas, con tripletas o con cualquier estructura de objetos con nombre.
Comprimidos	`.zip`, `.tar`, `.gz`, `.bz2`, `.xz`	Uno o varios grafos dentro, en cualquiera de los formatos anteriores.

Si el fichero no tiene forma de grafo (un PDF, una foto, código fuente, una página web), Nexus-Office lo dice y propone llevarlo a la **Biblioteca**.

Qué se guarda de cada grafo

- **Nodos**: cada entidad del grafo, con su nombre (`name`, `label`, `title`, `rdfs:label`…) y su tipo (la etiqueta de Neo4j, el `rdf:type`, la columna `type`…).
- **Relaciones**: las aristas, con su nombre en minúsculas y con guiones bajos (`TRABAJA_EN` y `trabajaEn` quedan como `trabaja_en`). Si no tienen nombre, quedan como `relacionado_con`. El peso se conserva.
- **Atributos**: las propiedades de cada nodo (propiedades de Neo4j, literales RDF, columnas de una tabla, atributos de GEXF o GraphML) van a su **nota** como `clave: valor`. Los agentes los ven al hablar de esa entidad: si importas que el Teide tiene `altura: 3718`, el agente lo sabe al preguntarle por el Teide.
- **Nodos anónimos de RDF**: se resuelven para que el conocimiento quede entre entidades con nombre. `ex:Ana ex:vive [ ex:ciudad ex:Bilbao ]` queda como *Ana vive_ciudad Bilbao*.
- **Ámbitos**: el grafo de otro Nexus-Office mantiene lo que era general y lo que era de cada agente, si ese agente existe aquí.

Un grafo puede tener hasta 200.000 nodos y la subida, hasta 300 MB. Un grafo de 100.000 relaciones se lee y se inserta en pocos segundos.

Para desarrolladores

- La lectura está en `nexus/memory/kg_formats.py`: `parse_bytes(datos, nombre)` detecta el formato con `detect()` y devuelve `{"format", "nodes", "edges"}`. `parse_many([(nombre, datos), …])` une varios ficheros; las tablas comparten ids, de modo que `nodes.csv` y `edges.csv` forman un solo grafo.
- La fusión está en `nexus/memory/kg_io.py`: `insert_graph(grafo, leido, destino, organizador, agentes)` devuelve `nodes`, `new`, `merged`, `relations`, `skipped` y `by_scope`.
- Lo que dentro de un ZIP de documentos cuenta como grafo lo decide `looks_like_graph()`: las interpretaciones de último recurso (un JSON o YAML cualquiera, un esquema con sangría) llevan `generic: true` y no cuentan.
- API: `POST /api/kg/import` acepta `file` o varios `files`; `GET /api/kg/formats` da la lista de formatos. Ver API.md.

- Pruebas: `tests/test_graph_import.py`, con un ejemplo real de cada formato.

Biblioteca RAG y conversión URL → PDF

RAG (generación aumentada por recuperación)

- **Formatos de documento**:
- PDF, por páginas.
- Word (DOCX).
- OpenDocument: ODT, ODS y ODP.
- PowerPoint (PPTX) y Excel (XLSX), por diapositiva u hoja.
- EPUB, por capítulo.
- RTF, HTML, Markdown, TXT, CSV, JSON y YAML.
- Código fuente, con más de 80 extensiones.

Cada documento se copia en `data/rag/files/`, así que la biblioteca viaja con la carpeta. `GET /api/rag/formats` devuelve la lista exacta. - **Archivos comprimidos**: ZIP, RAR, 7z y TAR (`.tar`, `.tar.gz`/`.tgz`, `.tar.bz2`, `.tar.xz`). Ver la sección siguiente. - **Importación en lote**: suelta muchos ficheros a la vez en la Biblioteca, sueltos y comprimidos mezclados. Se envían en tandas de 20 a `POST /api/rag/import` (multipart, campo `files`). La respuesta trae `results` (uno por fichero, en el mismo orden), `ok`, `failed` y `seconds`. Si un fichero falla, se informa de él y los demás se importan igualmente. - **Troceado** por párrafos (~900 caracteres, solapamiento de 150) con número de página. - **Búsqueda híbrida**: `0,7 × similitud semántica + 0,3 × coincidencia de términos`. Usa el mismo motor de embeddings que la memoria (sentence-transformers › Ollama › hashing). - **En cada mensaje**, los 3 fragmentos más relevantes (si superan el umbral) se añaden al contexto del agente con su fuente; el modelo cita como `[n]`. Sin LLM, el núcleo integrado responde citando los fragmentos. - **Herramientas**: `rag_search`, `rag_list`, `url_to_pdf` y `crawl_docs`. - **Interfaz**: *Núcleo → Biblioteca RAG*: procesar URL, rastrear documentación, arrastrar documentos, buscar, abrir, mover de grafo y borrar.

Leer páginas como un navegador (2.2)

Muchas webs (tiendas, documentación hecha con React, Vue o Angular, paneles) llegan casi vacías y se pintan después con JavaScript. Para que la Biblioteca, *URL a PDF*, el rastreador y la herramienta web de los agentes lean lo mismo que tú:

- Nexus-Office busca un **Chrome, Edge, Chromium o Brave** en el equipo (Windows, macOS y Linux, también el `chrome-headless-shell` de Playwright) y lo abre **sin ventana** (`--headless --dump-dom`), dejando unos segundos para que la página ejecute su JavaScript. Se queda con el DOM final y lo procesa como cualquier HTML.
- **Cuándo**: en *Automático* solo si la página lo necesita (un `<div id="root">` o `__next` vacío, `<noscript>` que pide JavaScript, muchos scripts y poco texto). *Siempre como navegador* lo hace con todas; *Nunca* lo apaga. Se cambia en el menú **MEMORIA** de la barra superior o con `rag.browser` (`auto`, `siempre`, `nunca`).
- **Seguridad**: el navegador usa un perfil propio de Nexus-Office (no el tuyo), que se vacía de cookies y almacenamiento antes de cada lectura, y sale a internet por un proxy propio que **solo deja llegar a direcciones públicas** (nada de `localhost`, la red local ni los metadatos de la nube), salvo que se active `rag.allow_private_urls`. Como mucho hay dos navegadores a la vez.
- Si no hay navegador, o tarda más de 40 s, se usa la descarga normal y se avisa en la respuesta.
- Un administrador puede fijar uno concreto con `rag.browser_path`. `GET /api/rag/navegador` dice el modo y el navegador encontrado. La herramienta `web_fetch` de los agentes indica `"navegador": true` cuando lo ha usado.

Grafo general o grafo de un agente

El grafo de conocimiento tiene un **grafo general**, común a todos los agentes, y dentro de él un **subgrafo aislado por agente**.

- **Importar en → grafo general**: el documento lo consultan todos los agentes y su nodo entra en el grafo general.
- **Importar en → solo un agente**: solo ese agente lo encuentra (en el contexto de cada mensaje y con `rag_search`/`rag_list`), y el documento entra en su grafo individual. Su resumen y puntos clave, si es una URL o un rastreo, quedan como recuerdos privados de ese agente.
- El selector afecta a documentos sueltos, comprimidos, URL → PDF y rastreo. En la lista, cada documento muestra su grafo y un desplegable para moverlo; el nodo se mueve con él.

- Un agente puede importar para sí mismo con `url_to_pdf` o `crawl_docs` y `target: "agente"`. Por defecto se importa al general.
- Si se borra un agente, sus documentos pasan al grafo general.
- En la conversación, lo que se sabe del usuario (perfil, gustos, pautas, relaciones) va al grafo general. El resto lo reparte el organizador: el saber hacer de una especialidad va al subgrafo de ese agente y el conocimiento común, al general. Ver MEMORIA.md.
- *Ajustes → Gestor de conocimiento → Leer un ZIP* hace más que importarlo: inserta los grafos que traiga, agrupa los documentos por temas y los razona con el LLM. Ver MEMORIA.md.

Archivos ZIP, RAR, 7z y TAR con proyectos o documentación

Un archivo comprimido se puede adjuntar en el **chat** (botón 📎) o soltar en la **Biblioteca RAG**. Nexus-Office:

1. **Descomprime** el archivo de forma segura en `data/sandbox/proyectos/<nombre>/`. Si todo cuelga de una única carpeta raíz, se aplana. Si el nombre ya existe, se usa `<nombre>_2`, `<nombre>_3`…
2. **Indexa** en la biblioteca los documentos (PDF, DOCX, HTML, MD, TXT…) y el código y la configuración (más de 80 extensiones, además de `Dockerfile`, `Makefile`, `README`…) como **un único documento** del tipo `zip`, `rar`, `7z` o `tar`. Cada fragmento lleva delante la ruta del fichero (`[proyectos/x/src/main.py]`), así que las citas dicen de qué fichero sale. La primera página es la **estructura** del proyecto.
3. En el chat, el mensaje lleva una nota con la carpeta, para que el agente pueda explorarla con `file_list` (con `path`) y `file_read` y buscar con `rag_search`. Sin texto, pide un resumen del contenido.

Herramientas de descompresión:

Formato	Cómo se descomprime
ZIP y TAR	Con la biblioteca estándar de Python, sin instalar nada
RAR	Con el primer programa disponible: `unrar`, `7z` (con el códec RAR), `bsdtar` o `unar`
7z	Con `7zz`/`7z`/`7za`, `bsdtar` o `unar`

Si no hay ninguno, Nexus-Office dice qué instalar:

- Windows: 7-Zip o WinRAR. En Windows 10/11 también vale el `tar` del sistema.
- Ubuntu y WSL: `sudo apt install unrar 7zip`. El paquete `7zip` de Debian/Ubuntu no trae el códec RAR, así que para RAR instala `unrar` (o `unar`).
- macOS: `brew install sevenzip` o `brew install unar`.
- Termux: `pkg install unrar p7zip`.

Los programas externos descomprimen primero en una carpeta de cuarentena. Nexus-Office lee antes el listado del archivo para aplicar los mismos límites que al ZIP. Después revisa cada fichero, como con el ZIP, antes de moverlo a su carpeta. Los ficheros cifrados o dañados se descartan y se informa de ellos.

Seguridad y límites: - Rechaza rutas absolutas, con `..` o con unidad de Windows (*zip slip*), enlaces simbólicos y entradas cifradas. - Contra bombas ZIP: 20 000 ficheros, 1 GB descomprimido, 50 MB por fichero y ratio de compresión ≤ 200 (se comprueba el tamaño real al descomprimir, no el declarado). - No descomprime `.git`, `node_modules`, `__pycache__`, `venv`/`.venv`, `dist`, `build`, `target` ni similares. - No indexa (aunque sí descomprime) `.env`, claves (`.pem`, `.key`, `id_rsa`…), minificados, mapas, lockfiles ni `vendor`/`third_party/`. Tampoco binarios. Por fichero indexa como máximo 512 KB, y en total hasta 20 000 ficheros. - **Proyectos grandes por partes**: lee los ficheros en grupos de 200 y, cuando una parte llega a 6 millones de caracteres, sigue en la siguiente. Las partes son documentos correlativos de la biblioteca («Proyecto X · parte 2 (ficheros 201-400)»). Borrar la primera borra todas, junto con el archivo y su carpeta. - Al borrar el documento de la biblioteca se borran también el archivo y su carpeta descomprimida.

Rendimiento: procesador al máximo

- **Lectura en paralelo**: en una importación en lote, cada documento suelto se lee en un núcleo distinto, con un pool de procesos (`nexus/parallel.py`). Los archivos comprimidos se procesan uno tras otro, porque cada uno ya usa el disco a fondo.
- **Embeddings**: el motor *hashing* reparte entre núcleos los lotes a partir de 256 fragmentos. Ollama recibe lotes de 128 y sentence-transformers de 64. Los fragmentos se guardan en la base de datos con una sola transacción.
- **Ajustes** (`performance` en la configuración):
- `workers`: número de procesos. 0 = automático, uno por núcleo disponible.

- `crawl_connections`: descargas simultáneas al rastrear. 0 = automático, entre 4 y 8.
- `NEXUS_NO_PROCESSES=1` fuerza hilos en lugar de procesos. En Android se hace solo.

Rastreo (crawling) de un sitio de documentación

Para que Nexus-Office aprenda una documentación entera (un manual, la API de una librería, la guía de un producto), dale la página inicial:

- **Interfaz**: *Núcleo → Biblioteca RAG*, fila **RASTREAR**: URL, máximo de páginas (40 por defecto, hasta 100 000) y ámbito (*esta sección* o *todo el dominio + grafo*). Con *todo el dominio* el máximo no se aplica. Se ejecuta en segundo plano y aparece en el panel de rastreos.
- **Chat**: *«Rastrea la documentación de https://docs.python.org/3/tutorial/ y apréndela»*. Con *«todo el dominio»* recorre el sitio entero. Sin LLM, el núcleo integrado lo reconoce y lanza la herramienta.
- **Herramienta** `crawl_docs` (`url`, `max_pages`, `max_depth`, `scope`, `background`, `job`, `continue`), del agente Investigador. **API**: `POST /api/rag/crawl {url, max_pages, max_depth, scope, background, batch_pages}`.

Cómo recorre el sitio: 1. **Ámbito**: por defecto, la carpeta de la URL inicial (`https://docs.x.org/3/tutorial/` → solo `/3/tutorial/…`); con `scope=domain`, todo el dominio registrable con todos sus subdominios (ver *Todo el dominio, sin excepciones* más abajo). Si la portada redirige (`/docs` → `/docs/es/latest/`), el ámbito se toma del destino. 2. **Recorrido en anchura** hasta `max_depth` saltos (3 por defecto). Sigue también los enlaces del menú lateral, aunque ese menú no se indexa. 3. **robots.txt**: respeta `Disallow` y `Crawl-delay` (como máximo 5 s). - Si el sitio declara `Crawl-delay`, Nexus-Office va de una en una página. - Si no, descarga varias páginas a la vez (`performance.crawl_connections`, de 4 a 8 por defecto) por conexiones reutilizadas (keep-alive), con al menos 0,1 s entre peticiones. - El análisis de cada página (extracción y deduplicación) se reparte entre los núcleos. - Los resultados se aplican en el orden de descubrimiento, así que el resultado es idéntico al de un rastreo secuencial, pero varias veces más rápido. 4. **sitemap.xml** (el declarado en robots.txt o `/sitemap.xml`, también índices de sitemaps) como semilla, filtrado por el ámbito. 5. **Descarta**: otros dominios, imágenes, CSS/JS, descargas y PDF, búsquedas, inicios de sesión, `_sources`/`_static`, páginas `noindex`, canónicas ya leídas, contenido duplicado y páginas casi vacías. Quita los parámetros de seguimiento (`utm_*`, `fbclid`…) y los `#fragmentos`. 6. **Extracción** de cada página con el mismo extractor estructurado que URL → PDF.

Qué guarda: - **Un único documento** del tipo `web` en la biblioteca, una página por página rastreada. Cada fragmento lleva delante su URL (`[https://docs.x.org/3/tutorial/classes.html]`), así que las citas dicen de qué página sale. - Una **copia legible en Markdown** con índice (botón *ABRIR*), en `data/rag/files/`. - **Asimilación**: resumen y puntos clave a la memoria; el sitio al grafo de conocimiento (`ha_leido`, `trata_de`, `contiene` con los títulos de las páginas). - **Volver a rastrear** la misma URL y ámbito **actualiza** el documento: el anterior se borra al terminar el nuevo.

Todo el dominio, sin excepciones (2.0)

Con el ámbito **todo el dominio** Nexus-Office lee el sitio entero, tenga las páginas que tenga: - **Sin límite** de páginas ni de profundidad. `max_pages` y `max_depth` se ignoran. - **Todos los subdominios** del dominio registrable (`www.`, `blog.`, `docs.`…; también `.co.uk`, `.com.es` y similares), cada uno con su propio `robots.txt`. - **Documentos enlazados**: PDF, Word, LibreOffice (odt, ods, odp), PowerPoint, Excel, EPUB, RTF, texto, CSV y Markdown, hasta 60 MB cada uno, con los mismos lectores de la biblioteca RAG. Las URL que aparecen dentro del texto de un documento también se siguen. - **Páginas cortas** (desde 5 palabras): glosarios, fichas y avisos también entran. - **Sitemaps**: hasta 2000 ficheros de sitemap (índices incluidos). - Siempre en **segundo plano y por lotes**; si se cierra Nexus-Office, el trabajo **se reanuda solo** al volver a arrancar. - Solo quedan fuera lo que el sitio prohíbe en `robots.txt`, otros dominios, imágenes, CSS/JS, binarios y el contenido duplicado.

Grafo de conocimiento multicapa del dominio

Al terminar, el rastreo construye un **grafo propio del dominio** en `data/rag/crawls/<id>.kg.sqlite`, por capas: 1. **Fuentes**: dominio, secciones (subdominios y carpetas), páginas y documentos, con sus enlaces. 2. **Conceptos**: términos clave de cada página y documento, relacionados por coaparición y por parecido semántico (modelo de embeddings). 3. **Afirmaciones**: frases con datos (cifras, fechas, definiciones). Se **contrastan entre fuentes**: las que dicen lo mismo en páginas distintas quedan como *contrastada (N fuentes)*; las que dan cifras distintas sobre lo mismo, como *discrepante*, con cada versión y su fuente; el resto, como *fuente única*. También se cotejan con los otros documentos de la biblioteca. Los textos de plantilla que solo cambian un número (por ejemplo, «Capítulo 1…», «Capítulo 2…») no cuentan como discrepancia. 4. **Temas**: grupos de conceptos muy relacionados. 5. **Síntesis razonada**: con el modelo de IA activo, cada tema recibe un resumen y sus conclusiones, que razona solo a partir de las afirmaciones contrastadas y citando fuentes. Sin modelo, se queda en las capas anteriores.

En el panel de rastreos, cada trabajo de dominio muestra páginas, afirmaciones contrastadas y nodos, y tiene: - **CREAR GRAFO / REHACER GRAFO**: lo construye o lo vuelve a construir (por ejemplo, tras activar un modelo con razonamiento). - **⬇ ZIP**: descarga el **paquete de conocimiento** (`graph.json` en formato nexus-kg, `conocimiento.md` legible y `manifest.json`). - **DAR A AGENTE**: importa el grafo en el subgrafo de un agente (o en el general) y añade el texto de conocimiento a su biblioteca, así que el agente **adquiere ese conocimiento** y lo cita. - El ZIP se puede importar en otro Nexus-Office con **IMPORTAR CONOCIMIENTO DE DOMINIO (.zip)** en la *Biblioteca RAG* (va al grafo elegido en *Importar en*) (o `POST /api/rag/domain-knowledge/import`, con el fichero y el campo `agent`).

API: `POST /api/rag/crawls/{id}/knowledge` (construir), `GET /api/rag/crawls/{id}/knowledge` (ZIP), `POST /api/rag/crawls/{id}/knowledge/import` `{agent}`.

Rendimiento medido: 5000 páginas se añaden al grafo en unos 13 s y el grafo se construye en unos 3 s, con unos 350 MB de memoria.

Dominios enteros: rastreo por lotes

Un dominio grande no cabe de una vez, así que se procesa en **lotes correlativos** que se guardan según terminan: - Si `max_pages` supera el tamaño de lote, el rastreo se convierte en un **trabajo por lotes**. El lote depende de la RAM: 50 páginas con menos de 4 GB, 100 en móviles o con menos de 8 GB, 200 con menos de 16 GB y 300 a partir de ahí. `rag.crawl_batch_pages` lo fija a mano. - Cada lote es un documento propio, ordenado: «Documentación: X · lote 003 (págs. 401-600)». El resumen y los puntos clave se hacen con el primero. - Tras cada lote se guarda el trabajo en `data/rag/crawls/<id>.json`: la cola de URLs pendientes en orden de descubrimiento, las visitadas y las huellas del contenido. Al **reanudar** (después de pausar, de reiniciar Nexus-Office o de un corte) sigue por la página siguiente, sin repetir ninguna y en el mismo orden. - **Panel de rastreos** en la Biblioteca RAG: progreso, lotes y cola pendiente de cada trabajo, con **PAUSAR**, **REANUDAR** y ✕ (borra el trabajo y, si quieres, sus documentos). Un trabajo en curso cuando Nexus-Office se cerró aparece como *interrumpido* y se reanuda igual. - **Chat**: «aprende todo el dominio https://docs.x.org» lanza el trabajo en segundo plano (hasta 100 000 páginas); «sigue rastreando https://docs.x.org» reanuda el último sin terminar. Desde la herramienta, `crawl_docs` con `job=<id>` o `continue=true`. - Volver a rastrear la misma URL sustituye todos los lotes antiguos al terminar el primer lote nuevo. - API: `GET /api/rag/crawls`, `POST /api/rag/crawls/{id}/pause`, `POST /api/rag/crawls/{id}/resume` y `DELETE /api/rag/crawls/{id}?docs=true`.

Seguridad y límites: solo direcciones públicas (se comprueba en cada petición y tras cada redirección; `rag.allow_private_urls: true` permite la red local), 3 MB por página, 6 millones de caracteres por documento o lote y 15 minutos por tanda (un trabajo por lotes sigue en la siguiente tanda). Las páginas que solo muestran contenido con JavaScript se omiten por tener poco texto. La herramienta tiene **riesgo medio** (accede a internet): con la política por defecto el usuario la aprueba en una tarjeta.

URL → PDF estructurado → conocimiento

Pide en el chat *«Procesa https://... y hazme un PDF»* (o usa *Procesar URL* en la Biblioteca):

1. **Descarga** (solo direcciones públicas; se bloquean IPs locales también tras redirecciones; máx. 8 MB).
2. **Extracción estructurada**: título, descripción, autor, fecha, idioma, sitio; jerarquía de secciones; párrafos, listas, tablas, bloques de código, citas; imágenes y enlaces. Prioriza `<article>`/`<main>` y descarta navegación, pies, formularios, banners de cookies, etc.
3. **Resumen y puntos clave** con el LLM (o resumen extractivo sin LLM).
4. **PDF maquetado**: portada, índice con números de página, marcadores PDF navegables, resumen, puntos clave, contenido, galería de imágenes incrustadas (hasta 8), enlaces y metadatos. Fuentes Unicode del sistema (Segoe UI/DejaVu/Arial/Roboto) con respaldo automático.
5. **Publicación** al usuario: tarjeta de PDF en el chat (`data/outputs/docs/`).
6. **Archivo en el RAG**: el PDF generado se indexa en la biblioteca.
7. **Asimilación**: el resumen y los puntos clave pasan a la memoria a largo plazo; el documento se añade al grafo de conocimiento (`ha_leido`, `trata_de`, `publicado_en`, `escrito_por`, `contiene`) y aparece en el avatar.

Medido en el equipo de desarrollo (CPU i3, Qwen3-VL 2B): artículo de Wikipedia de 4 540 palabras → PDF de 25 páginas, 16 secciones, 8 imágenes, 74 fragmentos RAG y 14 conocimientos en ~68 s.

La herramienta tiene **riesgo medio** (accede a internet): con la política por defecto el usuario la aprueba en una tarjeta.

Red cooperativa (LAN virtual por internet)

Varias instancias de Nexus-Office forman una **malla cifrada**: se ven como si estuvieran en la misma red local, aunque estén en casas o países distintos.

> Para usar **tu** Nexus-Office desde el móvil o el navegador fuera de casa, consulta Acceso remoto (TLS 1.3 y dispositivos emparejados). Esta red sirve para conectar **varias instancias de Nexus-Office** entre sí.

Cómo funciona

- **Transporte**: WebSocket en un puerto propio (8766 por defecto), separado de la interfaz web, que sigue escuchando solo en `127.0.0.1`.
- **Cifrado**: AES-256-GCM con una clave derivada por *scrypt* de la frase de red. Sin la frase no se puede leer ni inyectar tráfico; las conexiones que envían tramas inválidas se cierran.
- **Anti-repetición**: cada trama lleva id único y marca de tiempo (ventana de 5 minutos).
- **Malla**: los mensajes se reenvían con TTL y deduplicación, así que un nodo detrás de NAT solo necesita conectar **hacia fuera** a cualquier nodo accesible; a partir de ahí ve a todos.
- **IP virtual**: cada nodo obtiene una dirección estable `10.77.x.y` derivada de su id y de la red.

Servicios

Servicio	Control
Presencia (nombre, IP virtual, hardware, modelo, agentes)	siempre
Chat de la LAN	siempre
Inferencia remota: usar el LLM de otro nodo (`peer:NOMBRE`)	`coop.allow_remote_inference`
Agentes remotos	`coop.allow_remote_agents` (desactivado por defecto); sus herramientas usan supervisión reforzada
Memoria compartida (hechos)	`coop.share_memory` (desactivado por defecto)
Herramientas compartidas	`coop.share_tools`; llegan como **propuestas** que hay que revisar

Puesta en marcha

1. **Nodo anfitrión** (admin): *Red → Crear red*. Activa *Aceptar conexiones entrantes* y haz el puerto 8766 accesible (ver abajo). Escribe sus URLs públicas (`ws://IP:8766`) y pulsa *Invitación*.
2. **Resto de nodos**: pegan el código `NEXUS1:…` en *Unirse* y pulsan *Activar*.
3. En la lista de nodos: *Ping* y *Usar su IA* (por ejemplo, un portátil sin GPU usando el modelo grande del PC de sobremesa).

> El código de invitación **contiene la clave de cifrado**: compártelo solo con quien deba entrar en tu red.

Hacer accesible un nodo

- **Tailscale** o **ZeroTier** (recomendado): Nexus-Office detecta sus direcciones (100.64.0.0/10 y redes ZeroTier) y las muestra; usa `ws://100.x.y.z:8766`.
- Redirección del puerto 8766 en el router hacia el PC anfitrión.
- Cualquier túnel TCP/WebSocket (por ejemplo cloudflared) que apunte al puerto 8766. Solo hace falta **un** nodo accesible; los demás pueden estar detrás de NAT.

VPN cifrada para la app Nexus-Office VPN (WireGuard)

La app **Nexus-Office VPN** para Android (`Nexus-Office-2.2.0-android-vpn.apk`) llega al Nexus-Office de tu PC por un túnel **WireGuard** cifrado. Funciona igual en Windows, Linux, macOS y WSL, en casa y fuera de ella (datos móviles), sin publicar Nexus-Office en internet.

Aspecto	Cómo es
Cifrado	WireGuard: Curve25519, ChaCha20-Poly1305 y BLAKE2s, más una **clave precompartida distinta para cada móvil**
Qué viaja por el túnel	Solo el tráfico de la app Nexus-Office VPN y solo hacia el PC (`10.78.0.1/32`). El resto del móvil sigue por su red normal.
Red del túnel	`10.78.0.0/24`. El PC es `10.78.0.1` y cada móvil recibe la siguiente IP libre.
Puerto	UDP 51820, configurable. Es el único que hay que abrir en el router.
Claves	Las del PC están en `data/vpn/` (permisos 600). La clave privada de cada móvil solo aparece en su código y Nexus-Office no la guarda.
Acceso a Nexus-Office	Dentro del túnel, Nexus-Office sigue pidiendo la **clave de red**, y el código la incluye.

Puesta en marcha

Todo está en *Red COOP → VPN cifrada para la app Android*, en modo administrador.

1. **Activar VPN**. Elige el puerto UDP y activa el interruptor. Nexus-Office pasa a escuchar en todas las interfaces, así que **reinícialo**.
2. **Levantar el túnel en el PC**. Pulsa **LEVANTAR / RECARGAR TÚNEL**: - Si Nexus-Office tiene permisos de administrador o root, lo hace solo. - Si no, muestra los comandos de tu sistema con botón COPIAR. Con ⬇ **DESCARGAR nexusvpn.conf** obtienes la configuración del PC.

Sistema	Instalar	Levantar
Windows 10/11	`winget install WireGuard.WireGuard`	`wireguard.exe /installtunnelservice nexusvpn.conf` (PowerShell como administrador) y reglas de firewall UDP
Linux	`sudo apt install wireguard-tools` (dnf o pacman)	`sudo systemctl enable --now wg-quick@nexusvpn`
macOS	`brew install wireguard-tools` o la app WireGuard de la App Store	`sudo wg-quick up nexusvpn.conf`
WSL	`networkingMode=mirrored` en `%UserProfile%\.wslconfig` y `sudo apt install wireguard-tools`	`sudo wg-quick up nexusvpn` y una regla de firewall de Hyper-V para el UDP

3. **Fuera de casa**: abre el puerto UDP en el router. El botón **ABRIR UDP … EN EL ROUTER (UPnP)** lo intenta automáticamente; si no funciona, redirige el puerto a mano hacia el PC.
4. **Añadir un móvil**. Escribe un nombre y pulsa **GENERAR CÓDIGO VPN**. Sale un código `NEXUSV1:…`, que puedes copiar o descargar. - El código contiene la clave del móvil, por eso **solo se muestra una vez**. - También puedes descargar el `.conf` para la app oficial de WireGuard. - El túnel del PC se recarga para aceptar el móvil nuevo.
5. **En el móvil**: abre la app Nexus-Office VPN, entra en *Conectar por VPN* y pega el código. Android pide permiso para crear la VPN solo la primera vez.

El código lleva todas las direcciones del PC: IP de la Wi-Fi y, si lo marcas, la IP pública. La app prueba primero la última que funcionó, así que el mismo código sirve en casa y fuera.

Para retirar un móvil, pulsa **QUITAR** en la lista y recarga el túnel.

API (solo administrador)

Método	Ruta	Uso
GET	`/api/vpn`	Estado, móviles, plataforma e instrucciones por sistema
POST	`/api/vpn/enable`	`{enabled, port}`. Devuelve `restart_required` si Nexus-Office debe reiniciarse.
POST	`/api/vpn/peers`	`{name, lookup_public_ip}`. Devuelve `code` (`NEXUSV1:`), `client_conf` y `endpoints`.

Método	Ruta	Uso
DELETE	`/api/vpn/peers/{id}`	Quita un móvil
GET	`/api/vpn/server.conf`	Configuración del PC en formato `wg-quick`
POST	`/api/vpn/apply`	Levanta o recarga el túnel, o devuelve los comandos si faltan permisos
POST	`/api/vpn/upnp`	Abre el puerto UDP en el router

Comprobado

`packaging/test_vpn_e2e.sh`, ejecutado como root en Linux, monta un «PC» y un «móvil» en espacios de red separados:

- El PC levanta el túnel con la `nexusvpn.conf` que genera Nexus-Office.
- El móvil configura WireGuard con el texto exacto que produce la app (`WgConfig.java`).

La prueba comprueba la negociación WireGuard, que `/api/status` responde por el túnel y que sin la clave de red Nexus-Office devuelve 401.

Problemas frecuentes

- **«Sin respuesta del PC» fuera de casa**: el puerto UDP no está abierto en el router, o tu operador usa CG-NAT (IP pública compartida). Con CG-NAT usa Tailscale o ZeroTier, o el acceso remoto HTTPS a través de otro equipo.
- **WSL**: sin `networkingMode=mirrored`, el móvil no puede llegar a la VPN dentro de WSL.
- **Tras añadir o quitar móviles**, recarga el túnel del PC.

Acceso remoto por internet

Usa tu Nexus-Office desde cualquier lugar (datos móviles, otra Wi-Fi, otro país) con el móvil, la tablet o un navegador, cifrado de extremo a extremo y solo para los dispositivos que tú emparejes.

> **Alternativa con la app Android**: la **VPN cifrada WireGuard** (app Nexus-Office VPN). Solo hay que abrir un puerto UDP, Nexus-Office no queda publicado en internet y funciona igual con el PC en Windows, Linux o macOS. Ver VPN.md.

Cómo funciona

	Puerto local (sin cambios)	**Puerto remoto (nuevo)**
Dirección	`http://IP:8775`	`https://IP:8443` (configurable)
Para	este PC y la red de casa (con la clave de red)	internet
Cifrado	— (red de confianza)	**TLS 1.3**
Acceso	clave de red	**dispositivo emparejado** (llave propia, revocable)

El puerto remoto exige la llave de un dispositivo emparejado **en cada petición y en cada WebSocket**, venga de donde venga, también desde el propio PC. Así, ni un túnel ni un proxy pueden saltarse la protección. Sin emparejar solo se ve la página de emparejamiento, `/api/ping` y los ficheros de la interfaz, que son código público. Datos, memoria, imágenes generadas y API quedan cerrados.

El cifrado elegido y por qué

La red cooperativa usa AES-256-GCM con una clave fija derivada de la frase de red (scrypt). Para el acceso por internet desde móviles y navegadores hay una combinación mejor equilibrada entre rapidez y seguridad:

Pieza	**Qué se usa**	**Por qué**
Protocolo	**TLS 1.3 únicamente** (TLS 1.2 y anteriores rechazados)	Saludo de un solo viaje de ida y vuelta, sin algoritmos antiguos. Nativo en navegadores y en la app Android, así que no hace falta cifrado en JavaScript.
Intercambio de claves	**X25519MLKEM768**: híbrido post-cuántico (X25519 + ML-KEM-768 del NIST). Clientes antiguos: **X25519**	Claves nuevas en cada conexión (**secreto hacia adelante**): aunque algo se filtrara en el futuro, el tráfico grabado hoy no se puede descifrar, ni siquiera con un ordenador cuántico.
Cifrado de datos	**AES-GCM** (PC, con aceleración AES por hardware) o **ChaCha20-Poly1305** (móviles sin AES por hardware)	Cada dispositivo usa el más rápido para él: el servidor respeta la preferencia ChaCha20 del cliente.
Identidad del servidor	Certificado propio **ECDSA P-256**, **anclado por su huella SHA-256**	La app Android solo acepta tu servidor. No hace falta dominio ni autoridad de certificación.
Identidad del dispositivo	**Llave aleatoria de 256 bits** por dispositivo, obtenida con un **código de un solo uso** (60 bits, caduca en 10 min). El servidor solo guarda su huella	Revocable dispositivo a dispositivo. Una copia de los datos del servidor no permite entrar.
Fuerza bruta	8 fallos de código o de PIN en 15 minutos bloquean esa IP (no se aplica en el propio PC)	Adivinar un código o un PIN deja de ser viable.

Medido en el equipo de desarrollo, una vez abierta la conexión: **1,4 ms por petición**. El chat va por un WebSocket cifrado que permanece abierto, así que el cifrado no se nota.

Otros detalles: - La cookie del dispositivo es `HttpOnly`, `Secure` y `SameSite=Strict`. - Las respuestas incluyen `X-Frame-Options: DENY`, `nosniff`, `no-referrer` y `no-store`. - Al apagar, las conexiones se cierran en un máximo de 3 segundos, aunque un móvil se haya quedado sin cobertura.

Asistente del router (lo más fácil)

Ajustes → Asistente del router (o el botón **ASISTENTE DEL ROUTER** en *Red COOP → Acceso remoto*) te guía en cinco pasos:

1. **Acceso remoto en Nexus-Office**: lo activa si hace falta y te avisa si falta reiniciar.

2. **Tu router**: - Detecta la puerta de enlace, la IP y el adaptador de este equipo. - Identifica la marca cuando puede: Movistar, Vodafone, Orange/Jazztel, Digi, FRITZ!Box, TP-Link, ASUS, Netgear, Huawei, ZTE, Xiaomi, MikroTik, D-Link, Linksys o Sagemcom. - **Abre la configuración del router** en el navegador. Inicias sesión tú, con la contraseña de la pegatina.
3. **Abrir el puerto**: - Si el router admite UPnP, un clic. - Si no, el asistente indica dónde está la opción en el menú de tu marca (orientativo; varía según el firmware) y la **regla exacta** con botones para copiar: nombre, protocolo TCP, puerto externo, IP interna de este equipo y puerto interno. - También recomienda **reservar la IP** de este equipo en el DHCP del router (te da su MAC), para que la regla no deje de valer.
4. **Comprobar desde internet**: - Consulta tu IP pública (ipify) e intenta conectar a ella. También detecta si tu operador usa CGNAT. - Muchos routers no permiten esta prueba desde dentro de casa (*hairpin NAT*), así que la prueba definitiva es el móvil con datos móviles.
5. **Conectar la app Android**: genera el código `NEXUSR1:…` para pegarlo en *Conectar desde internet*.

El asistente solo **lee**: el título de la página del router y su descripción UPnP. Nunca inicia sesión ni cambia nada en el router. Los routers que responden con cabeceras no estándar o con TLS antiguo se detectan igualmente.

Puesta en marcha (modo administrador)

1. **Red COOP → Acceso remoto por internet**: activa *Permitir acceso desde internet* (puerto 8443 o el que elijas) y **reinicia Nexus-Office**. La etiqueta pasa a «escuchando en el puerto 8443».
2. **Haz que llegue desde internet** (una de estas opciones): - **Abrir puerto en el router (UPnP)**: Nexus-Office pide al router que redirija el puerto TCP a este PC y muestra la IP pública. *Cerrar puerto* lo deshace. - **A mano**: en el router, redirige el puerto TCP 8443 a la IP local de este PC. - **Tailscale o ZeroTier**: si ya los usas, no hace falta abrir nada. Nexus-Office muestra su dirección. - **Túnel** (Cloudflare Tunnel o similar): apunta el túnel a `https://localhost:8443`, nunca al puerto 8775.
3. Si el panel avisa de **CGNAT** (tu operador comparte la IP pública), abrir puertos no basta: usa Tailscale, ZeroTier o un túnel.
4. **Generar código**, con un nombre opcional para el dispositivo. El código sirve una sola vez y caduca a los 10 minutos.

Emparejar un dispositivo

- **App Android** (recomendado): *Conectar desde internet* → pega el código de acceso `NEXUSR1:…` (botón COPIAR del panel).
- El código incluye las direcciones, la huella del certificado y el código de un solo uso.
- La app ancla la huella, prueba las direcciones (internet, casa o Tailscale) y se empareja sola.
- Después recuerda las direcciones: con datos móviles usa la de internet y en casa la local.
- **Navegador**: abre el enlace `https://IP:8443/?pair=CÓDIGO`, o la dirección a secas y escribe el código.
- La primera vez el navegador avisará de que el certificado no es de confianza, porque es tu certificado propio.
- Antes de continuar, comprueba que la **huella SHA-256** que muestra el navegador (candado o aviso → certificado) coincide con la del panel. Si no coincide, no continúes.

Los dispositivos emparejados aparecen en el panel con su último acceso e IP. **Revocar** los desconecta al momento.

Avanzado

- *Regenerar certificado* cambia la huella: las apps tendrán que volver a emparejarse. Úsalo si sospechas que la clave privada se ha copiado (`data/remote/key.pem`).
- Configuración: `remote.enabled`, `remote.port`, `remote.host` (por defecto `0.0.0.0`).
- Datos: `data/remote/` (`cert.pem`, `key.pem`, `devices.json`, este último solo con huellas de las llaves).
- API: ver API.

Dispositivos: luces, reloj y avatar de bolsillo

Nexus-Office reparte su estado (en línea, escuchando, pensando, usando una herramienta, hablando, hecho, error) y lo que hace cada agente a tres dispositivos baratos que se conectan por Wi-Fi a tu ordenador. Todo se graba y configura **sin internet**: los binarios y el grabador vienen en el paquete.

Dispositivo	Placa	Qué hace	Binario (`firmware/bin/`)
NexusLED	Toda la familia ESP con Wi-Fi: ESP8266 (ESP-01/01S, ESP-12, NodeMCU, Wemos D1 mini), ESP32, S2, S3, C3, C5 y C6, con tiras WS2812, WS2812B o SK6812 (RGB y RGBW)	Luces que muestran el estado de Nexus (vúmetro con su voz al hablar), 23 efectos, 11 paletas, hasta 4 tiras y 1500 LED (600 en el ESP8266)	`nexusled-esp8266.bin` (1 MB), `-esp8266-4m`, `-esp8266-512k`, `-esp32`, `-esp32s2`, `-esp32s3`, `-esp32c3`, `-esp32c5`, `-esp32c6`
NexusWatch	ESP32-2424S012C-Y / (B): ESP32-C3 con pantalla redonda GC9A01 de 1,28" y táctil CST816S	Reloj con el avatar animado, qué hace cada agente y los modos	`nexuswatch-esp32c3.bin`
NexusPico	Raspberry Pi Pico W + pantalla de tinta Waveshare 2,13" V3 o V4 (la del pwnagotchi)	Avatar de bolsillo con caras al estilo pwnagotchi según el estado	`nexuspico-picow.uf2`

Grabar el firmware

Desde Nexus-Office: *Dispositivos → Grabar firmware*. Conecta la placa por USB, elige el dispositivo y el modelo de ESP32, escribe la Wi-Fi y pulsa **GRABAR**. Nexus graba el binario, le pone la Wi-Fi, su propia dirección y la clave de red, y el dispositivo aparece en *Conectados ahora*. Para el Pico W, conéctalo con **BOOTSEL** pulsado.

Nexus-Office graba con su propio grabador en C++ (nexus-flash), sin esptool ni Python: detecta el chip, elige el binario, lo graba, lo comprueba y le manda la configuración. Si el adaptador USB de la placa (CH340, CP210x o FTDI) no tiene driver, **INSTALAR DRIVERS** lo instala.

ESP-01 y placas sin reinicio automático: con un adaptador USB-serie de 3,3 V, une GPIO0 a GND y dale corriente; Nexus espera 20 s a que entre en modo de carga. Al terminar, suelta GPIO0 y vuelve a darle corriente. El ESP-01S y cualquier ESP8266 de 1 MB o más usan `nexusled-esp8266.bin`; el ESP-01 azul antiguo de 512 KB, `-esp8266-512k` (sin actualización por Wi-Fi); NodeMCU y Wemos pueden usar `-esp8266-4m`.

Con la orden `nexus-flash` (en `firmware/` de la carpeta de instalación o compilada desde `firmware/flasher`):

```
nexus-flash puertos
nexus-flash luces --wifi MiRed MiClave --leds 60            # GPIO 16 en ESP32, GPIO 2 en ESP8266
nexus-flash reloj --wifi MiRed MiClave --nexus 192.168.1.10:8775 CLAVE_DE_RED
nexus-flash pico  --wifi MiRed MiClave --nexus 192.168.1.10:8775 CLAVE_DE_RED
```

Con el script de Python (`firmware/flash.py`, en la edición Python):

```
python firmware/flash.py puertos
python firmware/flash.py luces --wifi MiRed MiClave --leds 60 --pin 16
python firmware/flash.py reloj --wifi MiRed MiClave --nexus 192.168.1.10:8775 CLAVE_DE_RED
python firmware/flash.py pico  --wifi MiRed MiClave --nexus 192.168.1.10:8775 CLAVE_DE_RED
python firmware/flash.py config reloj --wifi OtraRed OtraClave      # solo cambiar la configuración
```

Los dos reconocen el chip y eligen su `.bin`. La *clave de red* es la de *Red COOP → Conectar dispositivos*, y Nexus-Office tiene que escuchar en la red local.

A mano: `esptool --chip esp32s3 write_flash 0x0 nexusled-esp32s3.bin` (los `.bin` son imágenes completas y van en la dirección `0x0`) o la web https://espressif.github.io/esptool-js/. Para el Pico W, copia el `.uf2` a la unidad `RPI-RP2`. `firmware/bin/firmware.json` trae la versión y la huella SHA-256 de cada binario.

Sin el script, cada dispositivo sin Wi-Fi crea su propia red (`NexusLED-XXXX`, `NexusWatch-XXXX` con clave `nexuswatch`, `NexusPico-XXXX` con clave `nexuspico`) con una página de configuración en http://192.168.4.1.

Luces NexusLED

- *Dispositivos → Luces*: **BUSCAR** encuentra las luces de tu red (mDNS y UDP) o se añaden por IP. Para cada una: encender, brillo, efecto, paleta, color y si sigue el estado de Nexus.
- Conexión: DIN de la tira al GPIO 16 (en el ESP32-C3, por ejemplo el 8 o el 10; en el ESP8266 el GPIO 2, que es D4 en NodeMCU y Wemos y el único libre en el ESP-01) con una resistencia de 330 Ω, y GND común entre el ESP32 y la fuente. Más detalles en `firmware/nexusled/README.md`.

Reloj NexusWatch

- **Inicio**: el avatar de Nexus animado según lo que hace, con un anillo del color del estado.
- Desliza a los lados: **Agentes** (qué hace cada uno ahora; toca uno para ver su escena y su tarea), **Modos** y **Info** (Wi-Fi, IP y Nexus al que está conectado; mantén pulsado para volver al portal de configuración).
- Baja el brillo a los 30 s y se apaga a los 2 min; un toque lo enciende.

Avatar NexusPico

Nexus genera una imagen por estado con el avatar y una cara al estilo pwnagotchi. El Pico las descarga una vez, las guarda y después solo pregunta el estado cada 2 s (refresco parcial, sin parpadeo). Si cambias el nombre del asistente o el estilo (`dispositivos.pico.estilo`: `avatar` o `caras`), las vuelve a descargar. Las imágenes se ven en *Dispositivos*. Pines (Pico-ePaper-2.13): RST GP12, DC GP8, CS GP9, CLK GP10, DIN GP11, BUSY GP13.

Modos

Un modo cambia de una vez la voz, las reuniones, el estilo de respuesta, el perfil de hardware y el brillo de las luces. Se elige en *Dispositivos*, desde el móvil o tocando en el reloj, y se aplica en todos a la vez.

Modo	Qué hace
Normal	Voz y luces como siempre
Manos libres	Responde siempre en voz alta
Silencio	Sin voz y luces tenues
Concentración	Sin reuniones automáticas ni voz; respuestas breves
Rendimiento	Perfil de hardware *Rendimiento*
Ahorro	Perfil de hardware *Ahorro*
Noche	Luces apagadas y sin voz

Un modo nunca cambia la seguridad ni la red. Se pueden añadir modos propios en `dispositivos.modos`.

API

`GET /api/dispositivos`, `GET` y `POST /api/dispositivos/estado`, `GET /api/dispositivos/reloj`, `POST /api/dispositivos/modo {modo}`, `GET /api/dispositivos/pico/manifiesto` y `/api/dispositivos/pico/{estado}.bin|.png`, `GET /api/dispositivos/leds/buscar`, `POST`, `PUT` y `DELETE /api/dispositivos/leds/...`, `GET /api/firmware`, `POST /api/firmware/grabar` y `GET /api/firmware/trabajo/{id}`.

Qué no se ha podido probar sin el hardware

Todo compila en CI y la lógica está probada en el PC (emulador de NexusLED, pantalla del reloj dibujada en un fichero y el código del Pico contra el servidor), pero falta probarlo en las placas reales: los tiempos de la pantalla GC9A01 y el táctil CST816S del reloj, el refresco de la tinta (V3 y V4) del Pico W y la salida de las tiras en cada modelo de ESP32.